Tertiary Infotech Academy Pte Ltd

UEN: 201200696W

LEARNER GUIDE

For

Business Process Automation with Power Automate and Copilot Studio Agents

TGS Ref No: TGS-2022017524

Conducted by

Tertiary Infotech Academy Pte Ltd

UEN: 201200696W

Version 2.0

DOCUMENT VERSION CONTROL RECORD

Version Number	Effective Date of Release	Summary of Included Changes	Author
1.0	24 Jun 2026	Initial release — full 3-day, 17-lab learner guide.	Course Development Team
2.0	2 Jul 2026	WSQ revision — new course title, labs updated to the current Copilot Studio / Power Automate UI, Course Sandbox environment, WSQ cover page.	Course Development Team

TABLE OF CONTENTS

Right-click and choose “Update Field”, or press F9, to build the table of contents.

Welcome! This Learner Guide takes you **click-by-click** through every hands-on lab in the WSQ course **Business Process Automation with Power Automate and Copilot Studio Agents** (Course Code: TGS-2022017524). Over three days you go from your first Power Automate flow, to AI business agents in Microsoft Copilot Studio, to complete end-to-end automated workflows — and finish by building your own in a capstone project.

Work through the labs **in order**: each one builds on the skills of the lab before it. Whenever you see a **Checkpoint**, stop and confirm your flow or agent behaves as described before moving on. The **Common Errors & Quick Fixes** and per-lab **Troubleshooting** tables will get you unstuck fast.

Note: Course flow at a glance — Day 1: Workflow automation concepts + Power Automate (Labs 0-5). Day 2: Business agents in Copilot Studio (Labs 6-11). Day 3: End-to-end workflows + capstone (Labs 12-16).

Common Errors & Quick Fixes

Keep this handy — these are the issues learners hit most often, with the one-line fix:

Symptom	Cause	Fix
“Unauthorized” when sending email	Outlook connection expired or the account has no mailbox	Reconnect the Office 365 Outlook connection with a mailbox-enabled account; both connections must be green ✓
Approval fails: “valid users in the organization”	Approver typed as an external email, not a tenant user	Pick the approver from the people-picker dropdown (a real user in your tenant; yourself is fine for testing)
Date logs as literal text ‘utcNow()’	Expression typed into the field as text	Enter it via the fx / Expression editor so it becomes a coloured token
Excel cell shows ########	The column is only too narrow	Auto-fit the column — the value is fine
An unwanted ‘For each’ wraps your action	You inserted a list/array value into a single-value field	Use single-value fields (Outcome, trigger inputs); delete the For each and re-add a plain action
Agent can’t see its flow	Agent and flow are in different environments	Set both Copilot Studio and Power Automate to the same environment (Course Sandbox)

Day 1 — Foundations & Power Automate

Module 1: Introduction to Workflow Automation

Note: **Read this before the Day 1 labs.** It explains the "why" behind everything you'll build over the next three days. ~20 minutes.

By the end of this reading you will be able to:

- Explain what a business workflow is and which tasks are worth automating
- Tell the difference between an agent that only *talks* and a workflow that actually *does work*
- Use the four building-block terms — **trigger, actions, outputs, steps** — that recur in every single lab

---

1. What is business workflow automation?

A **business workflow** is a repeatable series of steps that gets work done. You already run dozens of them by hand every week. For example:

Note: *A customer emails an enquiry → someone records it in a spreadsheet → a salesperson is notified → they reply.*

Four small steps, but each one takes attention, and any of them can be forgotten, delayed, or done inconsistently.

Automation means letting software perform those steps for you — consistently, instantly, and the same way every time — instead of doing them by hand. The best candidates for automation share three traits:

Trait	What it looks like	Everyday example
Repetitive	Done the same way many times	Logging enquiries, sending confirmations
Rule-based	Clear "if this, then that" logic	If amount > $1,000, get approval
Time-consuming	Manual copying, chasing, re-typing	Re-keying form data into a spreadsheet

What you gain: fewer mistakes, faster turnaround, a complete record of what happened, nothing falling through the cracks — and people freed up for higher-value work that actually needs human judgement.

Note: **Rule of thumb:** if you find yourself doing the same clicking, copying, and emailing over and over, it is probably a workflow waiting to be automated.

In this course you automate workflows with **two** Microsoft tools that work as a pair:

- **Power Automate** — builds the automated steps (the **flows**) that do the work.

- **Copilot Studio** — builds AI **agents** that understand natural-language requests and feed clean, structured data into those flows.

You'll learn Power Automate first (Day 1), then agents (Day 2), then how to combine them (Day 3).

---

2. Standalone agents vs integrated workflows

It helps to be clear, from the very start, about the difference between an agent that just *answers* and a workflow that *acts*.

	Standalone agent	**Integrated workflow**
What it does	Chats and answers questions	Performs real actions across systems
Example	"What's your return policy?"	Logs the enquiry, emails the team, files a ticket
Powered by	Copilot Studio alone	Copilot Studio + Power Automate
Outcome	Information	Action + record + notification

A **standalone agent** is genuinely useful, but it only *talks*. Ask it a question, get an answer — the conversation ends and nothing changes in your business systems.

An **integrated workflow** connects that same agent to Power Automate, so the conversation actually *triggers work*: a record gets saved, an email gets sent, an approval gets started. The customer feels heard *and* the back-office task is already done.

```
Standalone agent:   User asks  →  Agent answers           (talk only)

Integrated:         User asks  →  Agent answers
                                    └→ triggers a flow →  log + email +
approval
                                                          (talk AND action)
```

That bridge between conversation and action is the heart of this course.

---

3. Workflow logic: the four building blocks

Every workflow you build — whether a simple flow or a full agent-driven process — is made of these four parts. Learn these terms now; you will use them in *every* lab.

Trigger — *what starts the workflow*

The single event that kicks everything off. Examples:

- An **email is received**
- A **file is uploaded** to a folder
- A **form is submitted**

- A **schedule** is reached (e.g. every morning at 9 AM)
- An **agent** calls the flow

Note: Every flow has **exactly one** trigger, and it does nothing until that trigger fires.

Actions — *what the workflow does*

The steps that run after the trigger, in order. Examples:

- **Send an email**
- **Add a row** to an Excel table
- **Start an approval** and wait for a decision
- **Post a message** to Teams
- **Create or update** a record

Outputs — *the data the workflow produces or passes along*

The pieces of information that move between steps. Examples:

- The **sender's email address** from a received email
- The **customer name and product** captured by an agent
- The **approval result** ("Approved" / "Rejected")

Outputs from one step become inputs to the next — in Power Automate this is called **dynamic content**, and it's how data flows down the chain.

Steps — *the ordered sequence as a whole*

Trigger + actions, arranged top-to-bottom, make up the **steps** of the workflow. Steps can also include:

- **Conditions** — branching with if/else logic (e.g. *if amount > $1,000, route to a manager*)
- **Loops** — repeat an action for each item in a list

Putting it together

```
TRIGGER            ACTIONS (steps)                        OUTPUTS
─────────          ──────────────────────────             ──────────
Email received  →  1. Read sender + subject          →  sender, subject
                   2. Add a row to Excel             →  logged record
                   3. Send notification to sales     →  confirmation
```

Read that left to right: an event happens, the flow performs a series of steps, and data (outputs) is produced and passed along the way. Keep this **Trigger → Actions → Output** mental model in your head — it is the spine of everything that follows.

---

4. How the three days fit together

Day	You build	New skill	What it gives you
Day 1	Flows in **Power Automate**	Triggers, actions, outputs	The "hands" that do work — email, Excel logging, approvals
Day 2	Agents in **Copilot Studio**	Structured capture, tools	The "brain & mouth" that talk to people and produce clean data
Day 3	Agents **+** flows together	Orchestration	Complete **end-to-end** business workflows, then your own

Day 1 is all about Power Automate. You'll master the trigger → action → output rhythm by building real flows you can run today.

Next: Module 2: Introduction to Power Automate

Module 2: Introduction to Power Automate

Note: Read this after Module 1 and before the Day 1 labs. ~15 minutes.

This module turns the **Trigger** → **Actions** → **Output** idea from Module 1 into the actual tool you'll use all day: **Power Automate**. By the end you'll recognise the flow types, the common triggers and actions, and the one thing that trips up every beginner — connections.

---

1. What is Power Automate?

Power Automate is Microsoft's automation platform. It lets you build **flows** — automated sequences of steps that run across Microsoft 365 and hundreds of other apps, with little or no code. You design a flow visually in a browser by stacking one **trigger** and one or more **actions** (exactly the building blocks from Module 1).

With Power Automate you can:

- React to events — an email arrives, a file is uploaded, a form is submitted
- Move and transform data between systems — Outlook, Excel, SharePoint, Teams, Dataverse, and more
- Run approvals, send notifications, and schedule recurring jobs
- Be called by a Copilot Studio **agent** as a tool (you'll do this on Day 2)

You build flows at **https://make.powerautomate.com**, inside the environment you set up in **Lab 0**.

Note: Low-code, not no-thought. You don't write code, but you do think like a designer: what starts the flow, what it does, and what data moves between steps.

Types of flow you'll build in this course

There are four flavours of flow. They differ only in *how they start* — once running, they all do the same kind of work.

Flow type	Started by	Example	Lab
Instant	A person clicking Run / a button	Send a confirmation email on demand	Lab 1–3
Scheduled	A clock/timetable (Recurrence)	Daily reminder at 9 AM	Lab 4
Automated	An event	New form response, new email, new file	Lab 5, Day 3
Agent flow	A Copilot Studio agent	Agent logs a request and notifies the team	Day 2–3

---

2. Common triggers

Every flow starts with **exactly one trigger** — the event that kicks it off. These are the ones you'll use most:

Trigger	Connector / name	Fires when…	Used in
Email received	Office 365 Outlook — *"When a new email arrives (V3)"*	Mail lands in a folder	Day 3, Lab 10
File upload	OneDrive / SharePoint — *"When a file is created"*	A document is dropped into a folder	Day 3, Lab 11
Form submission	Microsoft Forms — *"When a new response is submitted"*	Someone submits your form	Lab 5
Schedule	*"Recurrence"*	A timetable you define is reached	Lab 4
Manual	*"Manually trigger a flow"*	You press **Run**	Labs 1–3
Agent call	*"When an agent calls the flow"*	A Copilot Studio agent runs the flow as a tool	Day 2–3

Note: **Tip — build with a manual trigger first.** When learning a new flow, start with a **manual** trigger so you can press Run and perfect the actions. Once they work, swap in the real trigger (form, email, schedule). The actions stay exactly the same — only the start event changes.

---

3. Creating workflow actions

After the trigger, you add **actions** — the work the flow performs. These are the core actions you'll lean on throughout the course:

Send emails

Office 365 Outlook → Send an email (V2). Send confirmations, notifications, and digests. Use **dynamic content** (outputs from earlier steps) to personalise the subject and body. *(Lab 1)*

Create Excel entries

Excel Online (Business) → Add a row into a table (and *List rows present in a table*). Log every record into a spreadsheet **table** — this becomes your audit trail and single source of truth. *(Lab 2)*

Note: Excel actions only see data that lives inside a proper **Table** (Insert → Table), not loose cells. This is a classic first-time gotcha.

Notifications & approvals

- *Approvals → Start and wait for an approval.* Pause the flow until a person approves or rejects, then branch on the **Outcome**. *(Lab 3)*
- Notifications to **Teams** or email keep the right people informed at each step.

Actions are wired together with:

- **Dynamic content** — one step's output becomes the next step's input (Module 1's "outputs")
- **Conditions** — if/else branching to route the process

Note: **Heads-up on approvals:** the approver you pick must be a real **user in your Microsoft 365 tenant**. Approvals can't be sent to an outside personal email address — pick someone in your organisation (yourself is fine for testing).

A note on expressions (fx)

Sometimes a field needs a small calculation or formatting — today's date, trimmed text, a number comparison. Power Automate provides an **expression editor** (the **fx** button) for this. You'll only need a few simple ones; we'll point them out in the labs when they appear.

---

4. Connections — how Power Automate reaches your apps

Each connector (Outlook, Excel, Approvals, Forms…) needs a **connection** — a saved sign-in that authorises the flow to act on your behalf. The first time you use a connector, you'll **sign in and consent**.

This is the **number one source of "why won't my flow run?"** for beginners, so commit it to memory:

Connection state	What you'll see	What to do
Healthy	Green / connected	Good to go
Broken / expired	Red ⚠ "Reconnect"	Reconnect before running
Wrong rights	**"Unauthorized"** error	The account lacks rights (e.g. no mailbox) — fix or use a different account

Note: **The golden rule for every lab:** green connection = ready; red connection = reconnect first. An **Unauthorized** error almost always means: reconnect the connector or check that the signed-in account actually has permission for that action.

---

5. Anatomy of a flow (what you'll see in the designer)

```
[ TRIGGER ]        ← one event that starts the flow
    │
[ Action 1 ]       ← e.g. Get details / read data
    │
[ Condition ]      ← optional if/else branch
   ├── If yes → [ Action ]
   └── If no  → [ Action ]
    │
[ Action 2 ]       ← e.g. Send an email / Add a row
```

The build-and-verify loop is the same in every lab:

1. **Save** the flow.
2. **Test** it — manually, or by triggering the real event.
3. Review the **run history** — **green = success, red = error** — to confirm it worked or to debug.

Get comfortable with this loop today; you'll repeat it dozens of times across the three days.

Next: Lab 1: Automated Email Workflow

Lab 0: Environment Setup — Create Your Copilot Studio & Power Automate Accounts

Lab Title

Environment Setup — Create Your Microsoft 365, Power Automate, and Copilot Studio Accounts

Lab Objectives

By the end of this lab, you will be able to:

4. Obtain a Microsoft 365 **work or school** account that can use the Power Platform
5. Create a dedicated Power Platform **Sandbox environment** (with Dataverse) for this course
6. Sign in to Power Automate and Copilot Studio and point **both** at the same environment

7. Confirm Outlook and Excel (OneDrive) are available for the later labs
8. Run a verification checklist so you start Lab 1 with zero surprises
9. Understand the difference between a work/school account and a personal account

Prerequisites

- A web browser (Microsoft Edge or Google Chrome recommended)
- A mobile phone (used once for security verification)
- A credit card *(only if you create a new Business trial in Option B — it is **not** charged during the free month)*
- About 30–40 minutes

Note: **Tip — Which account should I use?** - **Option A — You already have a Microsoft 365 work/school account** (e.g. `name@company.com` provided by your organization). Try this first — you may already have everything you need. Skip Option B and go straight to **Step 1**. - **Option B — You do NOT have a work/school account** (you only have a personal `@outlook.com` / `@gmail.com`). Power Automate and Copilot Studio require a *work or school* account, so you'll create one via a free **Microsoft 365 Business trial**. Do **Option B** first, then continue from Step 1.

Note: ⚠️ **Warning — Why not the Microsoft 365 Developer Program?** As of 2024, Microsoft restricted the free Developer Program E5 sandbox to people with an active **Visual Studio Enterprise or Professional subscription**. If you sign up without one you'll see *"You don't currently qualify for a Microsoft 365 Developer Program sandbox subscription."* — so we **do not** use that path in this course. Use **Option B** instead.

Scenario

You work for **ACME Pte Ltd** and will spend the next three days automating ACME's business workflows. Before you can build anything, you need a clean place to work: a Microsoft 365 account, the Power Automate and Copilot Studio apps, and — importantly — **one shared environment** called **Course Sandbox** that both apps point to. Getting this right now means every later lab "just works."

Step-by-Step Guide

Option B (only if you have no work/school account): Create a free Microsoft 365 Business trial (~15 minutes)

This creates a brand-new *work* account such as `admin@yourname.onmicrosoft.com` with Microsoft 365 (Outlook, Excel, OneDrive, SharePoint) — exactly what Power Automate and Copilot Studio need. Skip this entirely if you already have a work/school account.

10. Open a browser and go to **https://www.microsoft.com/microsoft-365/business** (or search "Microsoft 365 Business Standard free trial").

11. Choose **Microsoft 365 Business Standard** and select **Try free for 1 month**.
12. Enter an email address to start. When prompted, choose **Set up account** / **Create a new account**.
13. Fill in your details (name, business name — you may use your own name, country, phone for verification).
14. Create your **sign-in details**: a username and a domain, giving you something like `admin@yourname.onmicrosoft.com`. **Write these down** — this is the account you'll use for the entire course.
15. Verify with the code sent to your phone.
16. Add a **payment method** (credit card). You are **not charged during the 1-month free trial**.
17. Wait 1–2 minutes for provisioning. You now have a Microsoft 365 work tenant.

Note: **Tip:** In a classroom, your trainer may provide a ready-made account. Ask before creating a trial so you don't duplicate accounts.

Note: ⚠ **Warning — Avoid surprise charges.** If you don't intend to keep the trial, set a calendar reminder and cancel **before the renewal date** in the **Microsoft 365 admin center** → **Billing** → **Your products**.

Step 1: Sign in to the Microsoft 365 portal (~5 minutes)

18. Go to **https://www.office.com** (or **https://m365.cloud.microsoft**).
19. Select **Sign in** and enter your **work/school account** (Option A) or your new **Business trial account** (Option B), then your password.
20. If this is your first sign-in, you may be asked to set up multi-factor authentication (MFA). Follow the prompts using your mobile phone.
21. Once signed in, you should see the Microsoft 365 home page with app tiles (Outlook, Word, Excel, etc.).
22. Open **Outlook** (click the Outlook tile) and send yourself a quick test email to confirm it works — you'll rely on Outlook in Lab 1.
23. Open **Excel** and create a blank workbook to confirm it saves to **OneDrive** — you'll rely on this in Lab 2. You can discard the test workbook afterwards.

Note: ⚠ **Warning — No Outlook/Excel tiles?** Your account may not have a Microsoft 365 license. The *Send an email* action in Lab 1 fails with **"Unauthorized"** when the account has **no mailbox**. Ask your IT administrator to assign a license, or use the Business trial account from Option B (which always has a mailbox).

Step 2: Create your "Course Sandbox" environment (~7 minutes)

An **environment** is a container that holds your flows, agents, and data. For this course we'll create a dedicated **Sandbox** environment with **Dataverse** turned on, so Day 2/Day 3 Copilot Studio agents have a database to use.

24. Open a new tab and go to the **Power Platform admin center**: **https://admin.powerplatform.microsoft.com**.
25. Sign in with the **same account** from Step 1.
26. In the left menu, select **Manage** → **Environments**.
27. Select **+ New** (top of the page).
28. Fill in the **New environment** panel: — **Name:** `Course Sandbox` — **Type: Sandbox** — **Region:** your nearest region (e.g. Asia, Singapore) — **Add a Dataverse data store: Yes** ← important
29. Select **Next**, accept the defaults (language English, currency your local currency) and set **Security group** to **None** (the field is required — *None* means open access), then select **Save**.
30. Wait 1–3 minutes. The environment appears in the list with status **Ready**. Refresh if needed.

Note: **Tip:** If your tenant blocks creating environments (some organizations restrict this), use the existing **default** environment instead — just remember to select that *same* environment in both Power Automate and Copilot Studio in the next steps.

Step 3: Sign in to Power Automate and select the environment (~5 minutes)

Power Automate is where you build the automated workflows (called **flows**).

31. Open a new tab and go to **https://make.powerautomate.com**.
32. Sign in with the **same account**.
33. The first time, you may be asked to **select your country/region** — choose the correct one and select **Get started**.
34. You'll see the Power Automate home page with a left-hand menu: **Home, Create, Templates, Learn, My flows**, plus pinned items such as **Approvals** and a **More** menu (items like **Connections** live under **More**).
35. Look at the **top-right corner** — there is an **Environment selector**. Click it and choose **Course Sandbox** (the one you created in Step 2). All your flows will be built here.
36. Select **My flows** in the left menu — it will be empty for now. That's expected.

Note: ⚠ **Warning:** The environment selector is the single most common source of "where did my flow go?" confusion. If you build a flow in the wrong environment, it simply won't appear when you switch. Always confirm **Course Sandbox** is showing top-right before you build.

Note: **Tip — Free Power Automate:** A Power Automate use-rights plan is included with most Microsoft 365 licenses, which is enough for this course. If prompted, you can also start a **free 90-day trial** of Power Automate Premium.

Step 4: Sign in to Copilot Studio and match the same environment (~5 minutes)

Copilot Studio is where you build the AI **agents** (used on Day 2 and Day 3).

37. Open a new tab and go to **https://copilotstudio.microsoft.com**.
38. Sign in with the **same account** again.
39. If prompted, select your **country/region** and select **Start free trial** (or **Try free**). This activates a **30-day Copilot Studio trial** at no cost (when it expires you can extend it once by another 30 days).
40. Wait for the workspace to load. You'll see the Copilot Studio home page with options to **Create** an agent.
41. Look at the **Environment selector** in the **top-right corner** and choose **Course Sandbox** — the **same** environment you selected in Power Automate.
42. Do **not** create an agent yet — you'll do that in a later lab. For now, just confirm the page loads in the correct environment.

Note: ⚠️ **Warning — Both tools MUST use the same environment.** Your agents (Copilot Studio) and your flows (Power Automate) can only call each other when they live in the **same** environment. If Power Automate shows *Course Sandbox* but Copilot Studio shows *Default* (or vice-versa), they cannot connect. Fix it now by clicking the top-right selector in each tool and choosing **Course Sandbox**.

Step 5: Verify your full setup (~5 minutes)

Run this quick checklist. Each item should already be true if the steps above succeeded.

#	Check	Where
1	I can sign in and see app tiles	https://office.com
2	I can open Outlook and send myself an email	Outlook
3	I can open Excel and it saves to OneDrive	Excel / OneDrive
4	My **Course Sandbox** environment shows status **Ready**	https://admin.powerplatform.microsoft.com
5	Power Automate home page loads and **Course Sandbox** is selected	https://make.powerautomate.com
6	Copilot Studio loads, my trial is active, and **Course Sandbox** is selected	https://copilotstudio.microsoft.com
7	Power Automate and Copilot Studio show the **same** environment	Top-right selector in both

If all seven are checked, your environment is ready.

Checkpoint

You should now have:

- ✅ A working Microsoft 365 **work/school** account (Option A or B)
- ✅ A Power Platform **Sandbox** environment named **Course Sandbox** with **Dataverse = Yes**, status **Ready**
- ✅ Power Automate open with **Course Sandbox** selected top-right
- ✅ Copilot Studio open (trial active) with **Course Sandbox** selected top-right
- ✅ Outlook and Excel (OneDrive) confirmed working

Troubleshooting

Problem	Solution
"You can't sign in here with a personal account"	Power Platform needs a *work/school* account. Use **Option B** to create one via the Business trial.
"You don't currently qualify for a Microsoft 365 Developer Program sandbox subscription"	The Developer Program now requires a Visual Studio subscription. Use **Option B** (Business trial) instead.
+ New environment button is greyed out / missing	Your tenant restricts environment creation. Ask an admin, or use the **Default** environment and select it in both tools.
Environment created but stuck on **Preparing**	Wait 2–3 minutes and refresh the Environments list; provisioning Dataverse takes a moment.
Power Automate says "no environment"	Refresh, re-select your region, then pick **Course Sandbox** in the top-right selector.
Copilot Studio "Start free trial" button missing	You may already have a license — just proceed. Otherwise sign out and back in.
Different environment shows in each tool	Click the environment selector (top-right) in **both** tools and choose **Course Sandbox**.
No Outlook/Excel tiles	Your account lacks a Microsoft 365 license (and possibly a mailbox) — ask IT or use the Business trial account (Option B).

Key Takeaways

- Power Automate and Copilot Studio both need a **work/school** account — personal accounts won't work.
- The **Microsoft 365 Developer Program** is no longer a free path; use a **Business trial** if you need an account.
- An **environment** is the container for your work; this course uses one named **Course Sandbox** (Sandbox type, Dataverse = Yes).
- The **#1 setup mistake** is having the two tools on **different environments** — always verify the top-right selector matches in both.

Duration

~30–40 minutes

Next Steps

Read Module 1: Workflow Automation Concepts and Module 2: Introduction to Power Automate, then proceed to Lab 1: Automated Email Workflow.

Lab 1: Automated Email Workflow

Lab Title

Build an Automated Email Workflow in Power Automate

Lab Objectives

By the end of this lab, you will be able to:

43. Create a flow from a blank canvas in Power Automate's new designer
44. Configure a **manual trigger** with a text **input**
45. Add a **Send an email (V2)** action and create an Office 365 Outlook **connection**
46. Use **dynamic content** to insert the input value into the email body
47. **Save**, then **Test** → **Run** the flow and confirm the email is delivered
48. Read the **run history** to verify each step succeeded

Prerequisites

- Completed Lab 0 (accounts ready)
- Signed in at https://make.powerautomate.com with **Course Sandbox** selected (top-right)
- Outlook working with your account (a mailbox-enabled account)

Scenario

At **ACME Pte Ltd**, every customer enquiry should get an instant, personalized "thank you" reply so customers know they've been heard. In this lab you build the simplest possible automation — a flow you start by hand that emails a personalized confirmation. It teaches the core pattern you'll reuse all course long: **Trigger** → **Action** → **Output**.

Step-by-Step Guide

Step 1: Start a new flow (~5 minutes)

49. Go to **https://make.powerautomate.com** and sign in.
50. Confirm the **Environment selector** (top-right) shows **Course Sandbox** — the environment from Lab 0.
51. In the left menu, select **Create**.
52. Under "Start from blank", select **Instant cloud flow**.

Note: **Tip:** An **instant** flow is started manually by clicking a button — perfect for learning and testing. Later labs use automatic triggers (a form submission, a new email, etc.).

53. In the dialog box: — **Flow name:** `Lab 1 - Send Confirmation Email` — **Choose how to trigger this flow:** select **Manually trigger a flow**

54. Select **Create**. The new designer opens with one step already on the canvas: the trigger **Manually trigger a flow**.

Step 2: Add an input to the trigger (~5 minutes)

We'll let whoever runs the flow type a customer name, so the email can be personalized.

55. Select the trigger card **Manually trigger a flow** to open its configuration panel (it opens on the **left** side of the designer).
56. Select **+ Add an input**.
57. From the type list, choose **Text**.
58. Replace the default input name with `CustomerName`.

Note: **Tip:** This input becomes one of the trigger's **outputs** — a piece of data you can drop into later steps using dynamic content.

Step 3: Add the Send an email action (~10 minutes)

59. Below the trigger, select the **+** (plus) button, then **Add an action**.
60. In the search box, type **Send an email**.
61. Select the **Office 365 Outlook** connector, then choose the action **Send an email (V2)**.

Note: ⚠ **Warning:** Pick **Office 365 Outlook**, not Gmail, Outlook.com, or SMTP. Only Office 365 Outlook uses your course work account.

62. If this is your first use of the connector, Power Automate creates a **connection**: select **Sign in**, choose your course account, and approve. A green ✓ next to the connection means it's ready.
63. Configure the email fields in the action panel: — **To:** type your own email address (so you receive the test). — **Subject:** type `Thank you for your enquiry` — **Body:** build the message with the name inserted dynamically:
64. Click inside the **Body** field and type: `Hi ` (with a trailing space)
65. Select the **dynamic content** icon (the small lightning bolt) that appears in/next to the field.
66. From the list, under **Manually trigger a flow**, choose **CustomerName**. It appears as a coloured **token** (chip) in the field.
67. Click after the token and continue typing: `, thank you for reaching out to ACME Pte Ltd. We have received your enquiry and a team member will respond within 1 business day.`

Note: **Tip: Dynamic content** is how outputs from earlier steps get reused. The coloured `CustomerName` token is a placeholder — it's replaced with the real value when the flow runs.

Note: ⚠️ **Warning — don't paste stray characters.** Type the text yourself; don't copy backticks (` ``) or quotation marks from this guide into the field, or they'll appear literally in the email.

Step 4: Save and test (~5 minutes)

68. Select **Save** (top-right).

Note: **Tip:** There is **no separate "Send" button**. Running the flow *is* what sends the email — the *Send an email* action does the work. So the routine is always: **Save**, then **Test → Run flow**.

69. Select **Test** (top-right) → choose **Manually** → **Test** → **Run flow**.
70. Power Automate prompts for the input you defined: — **CustomerName:** type `Jane Tan`
71. Select **Run flow**, then **Done**.
72. Watch the run status — each step should show a **green check**.
73. Open **Outlook** and confirm the email arrived, personalized as "Hi Jane Tan".

Step 5: Review the run history (~5 minutes)

74. In the left menu, select **My flows**, then open **Lab 1 - Send Confirmation Email**.
75. Look at the **28-day run history** — you'll see your test run with a status.
76. Select the run to inspect each step's **inputs and outputs**. This is how you debug flows: a **green check** = success; a **red** ⚠️ = error (click it to read the message).

Checkpoint

You should now have:

- ✅ A flow named **Lab 1 - Send Confirmation Email** with a `CustomerName` text input
- ✅ An Office 365 Outlook connection showing a green ✓
- ✅ A successful test run (all steps green)
- ✅ A personalized email received in Outlook reading "Hi Jane Tan, …"

Troubleshooting

Problem	Solution
No "Send an email (V2)" action listed	Make sure you selected the **Office 365 Outlook** connector (not Gmail / Outlook.com / SMTP).
Action fails with **"Unauthorized"**	The Office 365 Outlook connection is broken/expired, **or** the signed-in account has **no mailbox**. Open the action's connection, **reconnect** with a mailbox-enabled account (green ✓).
Connection shows a red ⚠️	The connection needs to be re-authorized — select it and **reconnect** / sign in again.
Email not received	Check Junk/Spam; confirm the **To** address; re-

	run the test.
Can't find dynamic content	Click directly **inside** the Body field first, then open the lightning-bolt menu.
`CustomerName` shows as plain text, not a token	Delete it and re-insert it from the dynamic content list so it becomes a coloured token.
Clicked "Save" but no email arrived	Saving does not send. You must **Test** → **Run flow** — running the flow performs the action.

Key Takeaways

- Every flow follows the pattern **Trigger** → **Actions**.
- Trigger **inputs** become **outputs** that you reuse in later steps via **dynamic content** tokens.
- A connector needs a **connection**: green ✓ = ready, red ⚠ = reconnect. **"Unauthorized"** on *Send an email* means the Outlook connection is broken or the account has no mailbox.
- There is **no separate Send button — Save**, then **Test** → **Run** to make the actions happen.
- **Test** + **run history** are your tools for verifying and debugging.

Duration

~30 minutes

Next Steps

Proceed to Lab 2: Excel Data Logging Workflow.

Lab 2: Excel Data Logging Workflow

Lab Title

Capture Form Data and Log It into Excel with Power Automate

Lab Objectives

By the end of this lab, you will be able to:

77. Prepare an Excel workbook with a formatted **Table** in OneDrive
78. Build a flow with a **manual trigger** that collects Name, Email, and Message
79. Add an **Add a row into a table** (Excel) action and connect to your workbook
80. Map trigger outputs into Excel columns and stamp the date using an **fx expression**
81. **Save**, then **Test** → **Run** and verify each submission appears as a new row
82. Understand how to swap the manual trigger for a real Microsoft Forms trigger

Prerequisites

- Completed Lab 1
- Access to Excel via OneDrive (verified in Lab 0)

- Signed in at https://make.powerautomate.com with **Course Sandbox** selected (top-right)

Scenario

At **ACME Pte Ltd**, every customer enquiry must be recorded so nothing is lost. In this lab you build a flow that takes submitted details (name, email, message) and **logs each one as a new row** in an Excel workbook — a running enquiry register the whole team can see. You'll also timestamp each row automatically.

Step-by-Step Guide

Step 1: Create the Enquiry Log workbook with a Table (~10 minutes)

Power Automate can only read and write Excel data that is formatted as a **Table** — not loose cells.

83. Go to **https://office.com**, open **Excel**, and create a **New blank workbook**.
84. Rename it (click the file name at the top of the screen): `Enquiry Log`. It saves automatically to **OneDrive**.
85. In **row 1**, type these five column headers, one per cell (headers must be in row 1):
    — A1: `Date` — B1: `Name` — C1: `Email` — D1: `Message` — E1: `Status`
86. Select the header range **A1:E1**.
87. On the ribbon, select **Insert** → **Table**.
88. In the **Create Table** dialog, tick **My table has headers**, then select **OK**.
89. With the table selected, open the **Table Design** tab and set **Table Name** to `EnquiryTable`. This makes it easy to find in Power Automate.
90. Close the Excel tab — changes are saved to OneDrive automatically.

Note: ⚠ **Warning:** If you skip **Insert** → **Table**, your file will have data but **no Table**, and Power Automate's "Table" dropdown will be empty in Step 3. The Table is mandatory.

Note: **Tip:** Naming the table `EnquiryTable` is optional but strongly recommended — otherwise it appears as `Table1` and is harder to pick out later.

Step 2: Create the flow and trigger (~5 minutes)

We'll use a manual trigger with inputs to *simulate* a submitted form. (Microsoft Forms uses the same downstream pattern — see Step 5.)

91. Go to **https://make.powerautomate.com**, confirm **Course Sandbox** is selected top-right, then select **Create** → **Instant cloud flow**.
92. **Flow name:** `Lab 2 - Log Enquiry to Excel`
93. **Choose how to trigger this flow:** select **Manually trigger a flow**, then **Create**.

94. Select the trigger card to open its panel, then select **+ Add an input** three times, choosing **Text** each time, and name them: — `Name` — `Email` — `Message`

Step 3: Add the "Add a row into a table" action (~10 minutes)

95. Below the trigger, select the **+** (plus) button, then **Add an action**.
96. In the search box, type **Add a row into a table**.
97. Select the **Excel Online (Business)** connector → action **Add a row into a table**.
98. If prompted, **Sign in** to create the connection (green ✓ = ready).
99. Configure where your file lives: — **Location:** `OneDrive for Business` — **Document Library:** `OneDrive` — **File:** browse to and select **Enquiry Log.xlsx** — **Table:** select **EnquiryTable**
100. The action now shows one field per column. Fill them in: — **Date:** this must be an **expression**, not typed text. Click the **Date** field, then select the **fx** (Expression) editor. Type exactly:

```
formatDateTime(utcNow(),'yyyy-MM-dd HH:mm')
```

Then select **Add** / **OK**. The value should appear in the field as a single **coloured token** (chip).

- **Name:** click the field → dynamic content → choose **Name**
- **Email:** click the field → dynamic content → choose **Email**
- **Message:** click the field → dynamic content → choose **Message**
- **Status:** type the static text `New`

Note: ⚠ **Warning — fx, not typing.** The `formatDateTime(...)` expression **must** be entered through the **fx / Expression** editor so it becomes a coloured token. If you just type it into the Date field, Power Automate logs the literal text `formatDateTime(utcNow(),'yyyy-MM-dd HH:mm')` into the cell instead of the actual date.

Note: **Tip — UTC vs local time.** `utcNow()` returns the time in **UTC**, so a 4pm Singapore enquiry logs as 08:00. If you want Singapore local time, use this expression in the **fx** editor instead: `` `convertTimeZone(utcNow(),'UTC','Singapore Standard Time','yyyy-MM-dd HH:mm')`

Step 4: Save and test (~10 minutes)

101. Select **Save** (top-right).

Note: **Tip:** As in Lab 1, there's **no separate "submit" button** — running the flow *is* what writes the row. Always **Save**, then **Test** → **Run flow**.

102. Select **Test** → **Manually** → **Test** → **Run flow**.

103. Enter sample values when prompted: — **Name:** `Ahmad Rahman` — **Email:** `ahmad@example.com` — **Message:** `Interested in a bulk order of 50 units.`
104. Select **Run flow** → **Done**. Confirm every step shows a **green check**.
105. Open **Enquiry Log.xlsx** in Excel — a **new row** should appear with your values, a `New` status, and a timestamp in the Date column.
106. Run the test **2–3 more times** with different values to confirm each submission adds another row.

Step 5: (Optional) Connect it to a real Microsoft Form (~5 minutes)

To turn this into a true *form submission* workflow:

107. Create a form at **https://forms.office.com** with three questions: Name, Email, Message.
108. Build a **new** flow with the trigger **When a new response is submitted** (Microsoft Forms connector).
109. Add the **Get response details** action, then the same **Add a row into a table** action, mapping the Forms answers into the columns (and keep the same `formatDateTime(...)` fx expression for Date).

Note: **Tip:** This shows the power of triggers — swap the manual trigger for a Forms trigger and the *same* logging logic runs automatically on every real submission.

Checkpoint

You should now have:

- ✅ `Enquiry Log.xlsx` in OneDrive with a Table named **EnquiryTable** (headers Date, Name, Email, Message, Status in row 1)
- ✅ A flow named **Lab 2 - Log Enquiry to Excel** with Name/Email/Message text inputs
- ✅ A **Date** column populated by the `formatDateTime(...)` **fx** expression (a coloured token, not literal text)
- ✅ Several test rows logged, each with a timestamp and Status `New`

Troubleshooting

Problem	Solution
File or Table not listed in the action	Ensure the data is formatted as a **Table** (Insert → Table), saved in **OneDrive**, with headers in row 1.
Date cell shows the literal text `formatDateTime(utcNow(),...)`	You typed the expression instead of using **fx**. Delete it, click the **fx / Expression** editor, paste the expression, and confirm it becomes a coloured token.
Date cell shows `########`	This is **not** an error — the column is just too narrow. **Auto-fit** the column (double-click its

	right border) to see the value.
Times look wrong / off by hours	`utcNow()` is **UTC**. Use the `convertTimeZone(...,'Singapore Standard Time',...)` expression for local time.
Row added but Name/Email/Message blank	Re-map each column to the correct dynamic content token.
Stray backtick or quote appears in a cell	Don't paste characters from this guide. Type values yourself or use the fx editor.
Connection shows red ⚠	Re-authorize the **Excel Online (Business)** connection (sign in again) until it shows a green ✓.
Saved but no row appeared	Saving doesn't write rows. You must **Test** → **Run flow** — running performs the action.

Key Takeaways

- Power Automate reads/writes Excel **Tables** (Insert → Table), never loose cells.
- The **Date** must be entered via the **fx** editor so `formatDateTime(...)` becomes a token — typing it logs the literal text.
- `utcNow()` is **UTC**; use `convertTimeZone(...)` for Singapore local time.
- `########` in a cell means the column is too narrow — auto-fit it; it's not an error.
- The same logging action works behind **any** trigger — manual, Microsoft Forms, email, or a Copilot Studio agent.

Duration

~35 minutes

Next Steps

Proceed to Lab 3: Simple Approval Workflow.

Lab 3: Simple Approval Workflow

Lab Title

Build a Simple Approval Workflow with Power Automate

Lab Objectives

By the end of this lab, you will be able to:

110. Create an instant cloud flow with three manual trigger inputs (RequesterName, RequesterEmail, RequestDetails)
111. Add a **Start and wait for an approval** action and assign it to a real user in your tenant
112. Use a **Condition** to branch on the approval **Outcome** (Approved vs Rejected)
113. Send a different notification email in the **If yes** and **If no** branches
114. Test and verify both the Approve and Reject paths end-to-end

Prerequisites

- Completed Lab 1 and Lab 2
- Signed in at **make.powerautomate.com** in the **Course Sandbox** environment
- Your own signed-in account (it must exist in this tenant's directory) — you will be your own approver for testing

Scenario

At **ACME Pte Ltd**, a staff member submits a small purchase request. A manager must **approve or reject** it, and the requester must be **automatically notified** of the decision. You will build this with two essential building blocks: a human **approval** step and a **Condition** (branching). For testing, you will play both roles — requester and approver — using your own account.

Step-by-Step Guide

Step 1: Create the flow and add inputs (~7 minutes)

115. Go to **https://make.powerautomate.com**.
116. Top-right, confirm the environment selector reads **Course Sandbox**. If not, click it and switch.
117. In the left menu, click **+ Create**.
118. Under "Start from blank", click **Instant cloud flow**.
119. In the dialog: — **Flow name:** `Lab 3 - Simple Approval` — Choose the trigger **Manually trigger a flow**. — Click **Create**.
120. The designer opens with the **Manually trigger a flow** card. Click the card to open it, then add three inputs. For each, click **+ Add an input**, choose **Text**, and name them exactly: — `RequesterName` — `RequesterEmail` — `RequestDetails`

Note: **Tip:** Type the input names with no spaces, exactly as shown. You will reference them by these names later, and spaces make the tokens harder to find.

Step 2: Add the approval action and assign a real user (~10 minutes)

121. Below the trigger, click the **+** then **Add an action**.
122. In the search box type `approval` and select **Start and wait for an approval** (from the **Approvals** connector).
123. If prompted to sign in / create the **Approvals** connection, click **Continue** or **Sign in** and finish it. The connection must show a green check.
124. Configure the action: — **Approval type:** `Approve/Reject - First to respond` — **Title:** type `Approval needed:` then, with your cursor still in the box, open the dynamic content (lightning bolt) and insert **RequestDetails**. — **Assigned to:** click the field. A people-picker dropdown appears. **Start typing your own name or email** (the same account shown under your avatar at the top-right),

then **click your name in the dropdown** so it resolves to a person chip. — **Details:** type `Requested by ` then insert the dynamic token **RequesterName**.

Note: ⚠ **Warning:** The **Assigned to** field MUST be a real user that exists in THIS tenant's directory, and you must **pick the person from the dropdown** so it becomes a resolved chip. Do **not** type a free-text external address like `someone@othercompany.com`. If you do, the run fails with *"InvalidApprovalCreateRequest … Required field … valid users in the organization."* Use your own signed-in account and select it from the dropdown.

Note: **Tip: Start and wait for an approval** pauses the entire flow until the approver responds. The approver can respond by email, in Teams, or in the **Approvals** hub (left menu) at make.powerautomate.com.

Step 3: Add a Condition on the Outcome (~8 minutes)

125. Below the approval action, click **+** → **Add an action**.
126. Search `condition` and select **Condition** (from the **Control** connector).
127. Build the condition with exactly these three parts: — **Left value:** click the box, open dynamic content (lightning bolt), and insert **Outcome** (from the *Start and wait for an approval* step). — **Operator:** `is equal to` — **Right value:** type `Approve`

Note: ⚠ **Warning:** The right value must be exactly `Approve` with a capital **A** — this is the literal text the approval **Outcome** returns. `approve`, `Approved`, or `APPROVE` will never match, and every run will fall into the **If no** branch.

128. You now have two branches below the condition: **If yes** (approved) and **If no** (rejected).

Step 4: Add a notification email to each branch (~10 minutes)

In the "If yes" branch:

129. Click **Add an action** *inside the If yes branch*.
130. Search `send an email` and select **Send an email (V2)** (from **Office 365 Outlook**). Complete the connection if prompted (it must show a green check).
131. Configure: — **To:** insert the dynamic token **RequesterEmail** (a trigger input). — **Subject:** `Your request has been APPROVED` — **Body:** type `Hi ` , insert **RequesterName**, type `, your request "`, insert **RequestDetails**, type `" has been approved. You may proceed.`

In the "If no" branch:

132. Click **Add an action** *inside the If no branch*.
133. Select **Send an email (V2)** again.

134. Configure: — **To:** insert **RequesterEmail** — **Subject:** `Your request has been REJECTED` — **Body:** type `Hi ` , insert **RequesterName**, type `, unfortunately your request "`, insert **RequestDetails**, type `" was not approved. Please contact your manager for details.`

Note: ⚠️ **Warning:** Only use **single-value** dynamic fields here — the trigger inputs (**RequesterName**, **RequesterEmail**, **RequestDetails**) and, if you want it, the approval **Outcome**. Do **not** insert any approval **Responses** field. Power Automate auto-wraps an action in a **For each** loop the moment you insert a list/array value, which breaks this simple flow. If a **For each** appears around your email, delete it and re-add a plain **Send an email (V2)** using only single-value fields.

Note: **Tip:** Add the email **inside** each branch box, not below the whole Condition — otherwise it runs on both outcomes.

Step 5: Save and test BOTH paths (~10 minutes)

135. Top-right, click **Save**. Before testing, confirm **both** connections show a green check: **Approvals** and **Office 365 Outlook**.
136. Click **Test** → **Manually** → **Test** → **Run flow**, and enter: — **RequesterName:** `Siti` — **RequesterEmail:** your own email — **RequestDetails:** `New office chair - $120`
137. Click **Run flow** → **Done**. The flow **pauses** at the approval step (this is normal — it is waiting for you).
138. Respond to the approval. Fastest path: left menu → **Approvals** → **Received** tab → open the request → click **Approve** → **Submit**. (The email/Teams notification also works but can be slow or land in Junk.)
139. The flow resumes down the **If yes** branch. Confirm you receive the **APPROVED** email.
140. **Run the test again** with the same inputs, but this time **Reject** the approval. Confirm you receive the **REJECTED** email.
141. Open **My flows** → **Lab 3 - Simple Approval** → **Run history** and confirm the correct branch ran each time.

Checkpoint

- ✅ Flow **Lab 3 - Simple Approval** with manual trigger inputs RequesterName, RequesterEmail, RequestDetails
- ✅ **Start and wait for an approval** assigned to your own user (resolved as a person chip)
- ✅ **Condition** on **Outcome** `is equal to` `Approve`, with a Send an email in each branch
- ✅ Approve path → APPROVED email received; Reject path → REJECTED email received

Troubleshooting

Problem	Solution
Run fails: *InvalidApprovalCreateRequest … valid users in the organization*	The **Assigned to** value isn't a real tenant user. Clear it, type your own name, and **pick your account from the dropdown** so it becomes a person chip.
Flow stays "Running" forever	Expected — it's waiting for the approval response. Go to left menu → **Approvals** → **Received** and respond.
Condition always goes to **If no**	The right value must be exactly `Approve` (capital A) to match the **Outcome** text.
A **For each** loop wrapped your email	You inserted a list/array value (e.g. a **Responses** field). Delete the For each and re-add a plain **Send an email (V2)** using only single-value fields.
Send an email: **Unauthorized**	The Outlook connection is broken or the account has no mailbox. Reconnect **Office 365 Outlook** with a mailbox-enabled account; both connections must show green ✓ before running.
No approval email arrives	Check **Junk**; or just respond in the **Approvals** hub instead — it's more reliable.
Email actions empty / nothing sent	Make sure each **Send an email** sits *inside* its branch (If yes / If no), not after the Condition.

Key Takeaways

- **Start and wait for an approval** pauses a flow until a real human in your tenant decides.
- **Assigned to** must resolve to a person chip from the directory — never a typed external email.
- The approval **Outcome** (`Approve` / `Reject`) drives a **Condition**, which creates branching logic for different responses.
- Inserting a list/array value silently adds a **For each** — keep approval emails on single-value fields to avoid it.

Duration

~45 minutes

Next Steps

Proceed to Lab 4: Scheduled Trigger Workflow.

Lab 4: Scheduled Trigger Workflow

Lab Title

Build a Scheduled (Recurring) Workflow with Power Automate

Lab Objectives

By the end of this lab, you will be able to:

142. Create a **Scheduled cloud flow** that uses the **Recurrence** trigger
143. Configure the Interval, Frequency, **Time zone**, hours, and specific days
144. Send a **daily reminder email** automatically on a timetable
145. **Test/Run now** without waiting for the scheduled time
146. (Optional stretch) Read the Excel **EnquiryTable** and include a live **count** in the email

Prerequisites

- Completed Lab 1 (Send an email)
- *(For the optional stretch)* An Excel **Enquiry Log** workbook with table **EnquiryTable** from Lab 2
- Signed in at **make.powerautomate.com** in the **Course Sandbox** environment

Scenario

At **ACME Pte Ltd**, some work isn't triggered by an event — it just needs to happen **on a schedule**. Every **weekday morning** the team should get a reminder to review new enquiries. You will build a flow that runs automatically using the **Recurrence** trigger, so no one has to start it.

Note: **Tip:** Trigger types so far — **manual** (Labs 1–3, you press Run) and **scheduled** (this lab, runs by the clock). Lab 5 adds an **event** trigger (a form submission).

Step-by-Step Guide

Step 1: Create a scheduled flow (~6 minutes)

147. Go to **https://make.powerautomate.com**.
148. Top-right, confirm the environment selector reads **Course Sandbox**. If not, click it and switch.
149. In the left menu, click **+ Create**.
150. Under "Start from blank", click **Scheduled cloud flow**.
151. In the dialog: — **Flow name:** `Lab 4 - Daily Enquiry Reminder` — **Starting:** today's date and any time — **Repeat every:** `1` and **Week**
152. Click **Create**. The designer opens with a **Recurrence** trigger already added.

Step 2: Fine-tune the schedule (~10 minutes)

153. Click the **Recurrence** trigger card to open its configuration panel.
154. Set the basics: — **Interval:** `1` — **Frequency:** `Week`

Note: **Tip:** Frequency must be **Week** (not Day) — the **On these days** option in the next step only appears with a weekly frequency.

155. Under **Advanced parameters**, select and set: — **Time Zone:** `(UTC+08:00) Kuala Lumpur, Singapore` — **On These Days:** tick **Monday, Tuesday, Wednesday, Thursday, Friday** (leave Saturday and Sunday unticked) — **At These Hours:** `9` — **At These Minutes:** `0`
156. The schedule now reads: **every weekday at 9:00 AM Singapore time**.

Note: ⚠ **Warning:** You MUST set the **Time zone**. Without it, the Recurrence trigger uses **UTC**, so a "9" would fire at 9:00 UTC = 5:00 PM Singapore — the wrong local time. This is the same UTC-vs-local trap from Lab 2.

Step 3: Send the reminder email (~8 minutes)

157. Below the Recurrence trigger, click **+** → **Add an action**.
158. Search `send an email` and select **Send an email (V2)** (from **Office 365 Outlook**). Complete the connection if prompted (it must show a green check).
159. Configure: — **To:** your team's address (use your own mailbox for testing) — **Subject:** `Daily reminder: review new enquiries` — **Body:**

```
Good morning,
This is your daily reminder to review and follow up on new enquiries
in the Enquiry Log. Please action any items marked "New".
```

160. Top-right, click **Save**.

Note: ⚠ **Warning:** If you hit an **Unauthorized** error, the Outlook connection is broken or the account has no mailbox. Reconnect **Office 365 Outlook** with a mailbox-enabled account (see Lab 1). The connection must show a green ✓ before running.

Step 4: Test the flow now (~5 minutes)

You don't have to wait until 9 AM — you can run it on demand to check it works.

161. Top-right, click **Test** → **Manually** → **Test** → **Run flow** → **Done**.
162. Confirm every step shows a green check and the reminder email arrives in your inbox.
163. From now on the flow also runs **automatically** on its schedule.

Note: **Tip:** A scheduled flow only fires on its timetable in real life — **Test** → **Run flow** lets you verify it immediately instead of waiting for 9 AM tomorrow.

Step 5 (Optional stretch): Include a live count from Excel (~15 minutes)

Make the reminder smarter by counting how many enquiries are logged in **EnquiryTable**.

164. **Above** the Send an email action, click **+** → **Add an action**.
165. Search `list rows` and select **List rows present in a table** (from **Excel Online (Business)**).

166. Configure the location: — **Location:** OneDrive for Business — **Document Library:** OneDrive — **File:** browse to **Enquiry Log** (the `.xlsx` workbook) — **Table:** `EnquiryTable`

167. Open the **Send an email (V2)** action again. Click into the **Body** where you want the count, then open the **fx** (expression) editor and enter exactly:

```
length(outputs('List_rows_present_in_a_table')?['body/value'])
```

- Click **Add / OK** so it becomes a **token** (highlighted chip), not plain text.
- Example line: `There are currently` *(token)* `enquiries logged.`

168. Click **Save**, then **Test** → **Run flow** again. The email now shows the live record count.

Note: ⚠️ **Warning:** The name inside `outputs('...')` must match your action's **internal name** exactly, with spaces replaced by underscores. If the expression errors, open the **List rows present in a table** action → **…** menu → check the action name, and adjust `List_rows_present_in_a_table` to match.

Checkpoint

- ✅ A **scheduled** flow **Lab 4 - Daily Enquiry Reminder** using the **Recurrence** trigger
- ✅ Configured for **weekdays at 9:00 AM** with **Time zone (UTC+08:00) Kuala Lumpur, Singapore**
- ✅ A reminder email sent on a successful **Test** → **Run flow**
- ✅ *(Optional)* A live record count pulled from **EnquiryTable**

Troubleshooting

Problem	Solution
Email arrives at the wrong time	Set **Time Zone** in the Recurrence trigger's **Advanced parameters** (e.g. (UTC+08:00) Kuala Lumpur, Singapore). Without it the schedule uses UTC.
Flow runs every day including weekends	In **On These Days**, tick only **Monday–Friday**.
On These Days option is missing	Set **Frequency** to **Week** — the days-of-week option only appears with a weekly frequency.
Send email **Unauthorized**	Reconnect **Office 365 Outlook** with a mailbox-enabled account; the connection must show green ✓.
`length(...)` expression error	Match the name inside `outputs('...')` to the **List rows** action's actual internal name (spaces become underscores).
Don't want to wait for the schedule	Use **Test** → **Manually** → **Run flow** to run it immediately.
Count shows as literal text, not a number	The expression wasn't added via the **fx** editor as a token. Re-enter it through **fx** and confirm it becomes a highlighted chip.

Key Takeaways

- The **Recurrence** trigger runs flows automatically on a timetable — no human starts them.
- Always set the **Time zone** so schedules fire at the right local time, not UTC.
- Use **Test** → **Run flow** to verify a scheduled flow without waiting for its scheduled time.
- Scheduled flows are ideal for digests, reminders, and clean-up jobs.

Duration

~30 minutes (45 with the optional stretch)

Next Steps

Proceed to Lab 5: Form Submission Workflow.

Lab 5: Form Submission Workflow

Lab Title

Capture Microsoft Forms Submissions to Email and Excel (Automatic Trigger)

Lab Objectives

By the end of this lab, you will be able to:

169. Create a **Microsoft Form** with three required questions and a shareable **URL**
170. Build a flow using the **When a new response is submitted** trigger (Microsoft Forms)
171. Add **Get response details** to read each answer (mapping the Response Id)
172. **Email** the submitted enquiry to your team
173. **Log** the same enquiry into the Excel **EnquiryTable** — all automatically on submit

Prerequisites

- Completed Lab 1 (Send an email) and Lab 2 (Excel table + logging)
- An Excel **Enquiry Log** workbook with table **EnquiryTable** (Date, Name, Email, Message, Status) from Lab 2
- Signed in at **make.powerautomate.com** in the **Course Sandbox** environment

Scenario

At **ACME Pte Ltd**, customers fill in an online enquiry form. The moment they **submit**, the workflow should **email the team** the new enquiry **and record it in Excel** — with no manual work. This is your first **automatic, event-driven** workflow (Labs 1–3 were manual, Lab 4 was scheduled). You'll also get a **form link** you can send to anyone.

Step-by-Step Guide

Step 1: Create the form in Microsoft Forms (~10 minutes)

174. Open a new browser tab and go to **https://forms.office.com** (it redirects to the current home, **https://forms.cloud.microsoft** — both work).
175. Sign in with the **same account** you use for Power Automate in the **Course Sandbox** tenant.
176. Click **+ New Form**.
177. Click the title at the top and enter: `Customer Enquiry Form`.
178. Add a short description, e.g. `Tell us about your enquiry and we'll respond within 1 business day.`
179. Add these three questions. For each, click **+ Add new**, choose **Text**, type the question, then toggle **Required** on: — `Full Name` → **Required** on — `Email` → **Required** on — `Your Message` → click the **…** on the question and choose **Long answer** for more space → **Required** on
180. The form saves automatically as you type.

Note: **Tip:** Keep the question titles exactly as shown — you'll match them to Excel columns later, and clear names make the dynamic tokens easy to find.

Step 2: Get the shareable form URL (~5 minutes)

181. Top-right of Forms, click **Collect responses** (older UI: **Share**).
182. Set the audience — for testing choose **Anyone can respond** (or your organization).
183. Copy the **link** shown (e.g. `https://forms.office.com/r/XXXXXXXX`).
184. **Save this URL** somewhere — it's the link you'd send to customers, and you'll use it to test at the end.

Step 3: Create the automated flow (~5 minutes)

185. Go back to **https://make.powerautomate.com** (confirm the environment is **Course Sandbox**).
186. Left menu → **+ Create** → **Automated cloud flow**.
187. **Flow name:** `Lab 5 - Form Submission to Email and Excel`.
188. In "Choose your flow's trigger", search `Forms` and select **When a new response is submitted** (Microsoft Forms).
189. Click **Create**.
190. On the trigger card, set **Form Id** → pick **Customer Enquiry Form** from the dropdown.

Step 4: Get the response details (~10 minutes)

The trigger only gives you a response **Id** — you need another action to read the actual answers.

191. Below the trigger, click **+** → **Add an action**.
192. Search `Forms` → select **Get response details** (Microsoft Forms).
193. Configure: — **Form Id:** pick **Customer Enquiry Form** (the same form) — **Response Id:** click the field, open dynamic content (lightning bolt), and insert **Response Id** (from the trigger)

Note: ⚠ **Warning: Get response details** is **required**. Without it, later steps only see an internal Id, not the real answers — and your email/Excel will be blank. Always map **Response Id** from the trigger, then use **this action's** outputs (Full Name, Email, Your Message) in every step that follows.

Step 5: Email the submission to your team (~10 minutes)

194. Click **+** → **Add an action**.
195. Select **Send an email (V2)** (Office 365 Outlook). Complete the connection if prompted (it must show a green check).
196. Configure (insert each value from **Get response details** dynamic content): — **To:** your team's address (use your own working mailbox for testing) — **Subject:** type `New enquiry from` then insert **Full Name** — **Body:**

```
A new enquiry was submitted via the form:
Name: [insert Full Name]
Email: [insert Email]
Message: [insert Your Message]
```

Note: ⚠ **Warning:** If you get an **Unauthorized** error, the Outlook connection is broken or the account has no mailbox. Reconnect **Office 365 Outlook** with a mailbox-enabled account (see Lab 1); the connection must show a green ✓.

Step 6: Log the submission to Excel (~10 minutes)

197. Click **+** → **Add an action**.
198. Select **Add a row into a table** (Excel Online (Business)). Complete the connection if prompted.
199. Set the location: — **Location:** OneDrive for Business — **Document Library:** OneDrive — **File:** browse to **Enquiry Log** (the `.xlsx` workbook) — **Table:** `EnquiryTable`
200. Map the columns: — **Date:** click the **fx** (expression) icon → enter exactly `formatDateTime(utcNow(),'yyyy-MM-dd HH:mm')` → click **Add / OK** so it becomes a **token** (a highlighted chip), never typed text — **Name:** dynamic content → **Full Name** — **Email:** dynamic content → **Email** — **Message:** dynamic content → **Your Message** — **Status:** type `New`
201. Top-right, click **Save**.

Note: ⚠ **Warning:** The **Date** value must be entered through the **fx** editor and become a token. If you type the formula as plain text, the cell stores the literal text `formatDateTime(...)` instead of a real date. (This is the same fix from Lab 2.)

Step 7: Test the whole workflow (~10 minutes)

202. In Power Automate, click **Test** → **Manually** → **Test**. The flow goes into a "waiting for trigger" state.
203. Open the **form URL** you saved in Step 2 (new tab or your phone).
204. Fill in the form — Full Name `Jane Tan`, Email `jane@example.com`, Your Message `Interested in 50 units` — and click **Submit**.
205. Within about a minute the flow triggers. Confirm: — Every step shows a green check in the run. — The **email** arrives with the submitted details. — A **new row** appears in **Enquiry Log** → **EnquiryTable** with a clean date, the answers, and **Status** `New`.
206. Submit the form a **second time** with different details and confirm another row and another email.

Note: **Tip:** Triggers can take up to a minute. If nothing happens, confirm you clicked **Test** *before* submitting — or just submit again, since a saved flow fires automatically on every real submission.

Checkpoint

- ✅ A **Customer Enquiry Form** with three required questions and a shareable URL
- ✅ Automated flow triggered by **When a new response is submitted**
- ✅ **Get response details** mapping the **Response Id** from the trigger
- ✅ Each submission → **notification email** to the team **and** a **new row** in **EnquiryTable** (Date via fx token)

Troubleshooting

Problem	Solution
Form not listed in the trigger	Sign into Power Automate with the **same account** that owns the form, in the **Course Sandbox** environment.
Answers show as IDs / blank	You must add **Get response details**, map **Response Id** from the trigger, and use **its** outputs (not the trigger's) in later steps.
Flow doesn't trigger after submit	Triggers can take up to a minute; confirm **Test** was started before submitting, or submit again (a saved flow fires automatically).
Date shows literal text	Enter the date through the **fx** editor so it becomes a token (see Lab 2).
Send email **Unauthorized**	Reconnect **Office 365 Outlook** with a mailbox-enabled account; the connection must show green ✓.
Add a row **Unauthorized**	Reconnect **Excel Online (Business)** with an account that can access the Enquiry Log workbook.

Key Takeaways

- A **Microsoft Forms** trigger turns a public form into an automatic workflow — no buttons to press.
- **Get response details** is required to read the individual answers behind the Response Id.
- The **Date** column is filled by an **fx** token (`formatDateTime(utcNow(),'yyyy-MM-dd HH:mm')`), never typed text.
- One submission can fan out to **multiple actions** (notify **and** log) — the core pattern for Day 3.

Duration

~50 minutes

Next Steps

You've completed Day 1 and all the core Power Automate trigger types (manual, scheduled, form, email). Proceed to Day 2 — Module 3: Business Agents Concepts.

Day 2 — Building Business Agents with Copilot Studio

Module 3: Building Business Agents with Copilot Studio

Note: Read this before the Day 2 labs. ~15 minutes.

On Day 1 you built the **hands** — Power Automate flows that send email, log to Excel, and run approvals. Today you build the **brain and mouth**: a Copilot Studio **agent** that talks to people in plain language and hands clean, structured data to those flows.

By the end of this reading you'll know what an agent is made of, how to make it produce *predictable* data (the make-or-break skill), and how an agent calls a flow as a tool.

---

1. What is a business agent?

A **business agent** is an AI assistant you build in **Copilot Studio**. People chat with it in natural language ("I'd like a quote for 100 units"), and the agent:

- **Understands** the request
- **Asks** for any missing details
- **Captures** the information as clean, structured data
- **Hands off** that data to a Power Automate flow to take action

```
  Day 1                          Day 2
┌─────────┐                   ┌─────────┐
│ FLOWS   │  the "hands"      │  AGENT  │  the "brain & mouth"
```

```
┌──────────┐                          ┌──────────────┐
│ do work  │  ◄──── calls ──────      │ talks &      │
└──────────┘                          │ captures     │
                                      └──────────────┘
```

The agent is the friendly front door; the flow is the engine room behind it.

---

2. The building blocks of a Copilot Studio agent

An agent is assembled from five kinds of building block. You'll meet each one across today's labs.

Block	What it is	Example	First seen
Instructions	Plain-language directions that shape the agent's behaviour and personality	"You are a sales assistant. Always collect name, company, product, and quantity."	Lab 6
Knowledge	Documents / sites the agent can answer from (RAG)	Product catalogue, FAQ	Lab 7
Topics	Conversation flows the agent runs when the user's message matches the topic's **description** (generative orchestration, the default) or its **trigger phrases** (classic)	A "New Sales Enquiry" topic the agent chooses when someone asks for a quote	Lab 9
Tools (formerly *Actions*)	Things the agent can *do*, including **Power Automate flows**	"Log enquiry to Excel" flow	Lab 8
Variables	Where captured answers are stored to pass onward	`customerName`, `product`, `quantity`	Lab 9

Note: **Knowledge = RAG.** When you upload documents, the agent uses **Retrieval-Augmented Generation**: it *retrieves* the relevant passages from your files and *generates* an answer grounded in them — so it speaks from your content, not the open internet. You'll set this up in Lab 7.

---

3. Prompt design for structured outputs

This is the single most important skill for connecting agents to workflows: getting the agent to produce **structured, predictable data** — not just free-flowing chat.

Why it matters. A flow needs clean, named fields. Compare what the agent might hand over:

The agent hands the flow…	Can the flow use it?
`"the guy from ABC wants some units maybe 100"`	❌ No — nothing to log reliably

`name=John, company=ABC, product=Widget, quantity=100`	✅ Yes — every field drops straight into a row

When the data is clean and structured, **every downstream step just works**. When it's messy, the whole workflow is unreliable.

Three techniques to get structured output

207. **Capture into named variables.** Use question nodes that store each answer in its own variable (`customerName`, `email`, `quantity`). This is the most reliable method, and you'll use it in **Labs 9 and 10**.
208. **Write precise instructions.** Tell the agent exactly what to collect and in what form: — > *"Collect these four items one at a time: customer name, company, product of interest, and quantity. Do not proceed until all four are provided. Confirm the details back to the user before finishing."*
209. **Ask AI to format the result.** When you want a generated summary, specify the format explicitly: — > *"Summarize the enquiry as: Customer: <name>; Company: <company>; Product: <product>; Qty: <quantity>; Notes: <notes>."*

Good-prompt checklist

- ✅ State the agent's **role and goal**
- ✅ List the **exact fields** to collect
- ✅ Specify the **output format**
- ✅ Say **what to do when information is missing**
- ✅ Keep **tone** instructions short and clear

Note: **Structured agent output = clean flow inputs.** This is the through-line of Day 2. If the agent captures tidy variables, the flow receives tidy inputs — and the handoff in section 4 works first time.

---

4. Connecting agents to Power Automate flows

Once the agent has captured the data, it calls a flow as a **tool**:

```
USER chats with AGENT  →  AGENT captures variables  →  AGENT calls FLOW (tool)
                                                          passing variables as
inputs
                                                              |
                                                              ▼
                                                FLOW runs actions (log, email,
approve)
                                                              |
                                                              ▼
                                                FLOW returns a result  →  AGENT
confirms to user
```

The key points:

- The agent **passes its variables** (outputs) into the flow's **inputs**.
- The flow does the work and can **return a value** — say, a reference number — for the agent to show the user.
- This is exactly the same flow-building you learned on Day 1. **Only the trigger changes** — to *"When an agent calls the flow."*

Note: **It's still Day-1 Power Automate underneath.** Same designer, same actions, same Save → Test → run-history loop. The agent simply replaces the manual "Run" button as the thing that starts the flow.

---

5. What you'll build on Day 2

- **Lab 6:** Your first agent — learn the interface, instructions, and testing.
- **Lab 7:** Add **Knowledge (RAG)** so the agent answers from your own documents.
- **Lab 8:** Add **Tools & Actions** so the agent can *do* things, not just answer.
- **Lab 9:** A **Sales Enquiry Assistant** that captures enquiries as structured data.
- **Lab 10:** A **Procurement Request** agent that triggers a Power Automate flow.
- **Lab 11: Automated Response Generation** — use AI prompts to draft professional replies.

Each lab adds one capability on top of the last, so by Lab 11 you'll have an agent that understands, retrieves, captures, acts, and writes.

Next: Lab 6: Create Your First Copilot Studio Agent

Lab 6: Create Your First Copilot Studio Agent

Lab Title

Create, Configure, and Test Your First Business Agent

Lab Objectives

By the end of this lab, you will be able to:

210. Create a new **blank agent** in Microsoft Copilot Studio and configure it yourself
211. Write clear **Instructions** that shape how your agent behaves
212. Identify the main building blocks of an agent (Instructions, Knowledge, Topics, Tools)
213. Add one **Knowledge** source so your agent can answer real questions
214. Test and iterate on your agent in the built-in **Test** pane
215. Publish your agent (optional) and understand what Channels are

Prerequisites

- Completed Lab 0 (Copilot Studio trial active)
- Read Module 3
- Signed in at https://copilotstudio.microsoft.com (same environment as Power Automate)

Scenario

You work in IT at **ACME Pte Ltd**, a small company. Staff keep emailing IT the same basic questions: opening hours, who to contact, how to reset things. You will build a friendly **Company Helpdesk** agent that answers these questions automatically.

This lab is your guided tour of Copilot Studio. You will learn the interface and the core idea that **Instructions are the heart of the agent** before you build data-capturing, action-taking agents in the labs that follow.

Note: **Tip:** An "agent" (sometimes still called a "copilot") is just an AI assistant you configure. You do not write code — you describe what you want in plain English, add some reference material, and test it in a chat window.

Step-by-Step Guide

Step 1: Confirm your environment (~3 minutes)

Before you build anything, make sure you are in the correct environment. This is the single most common cause of problems later in the course.

216. Go to **https://copilotstudio.microsoft.com** and sign in with your course account.
217. Look at the **top-right corner** of the screen. You will see an **environment selector** (a small label showing the current environment name, often with a globe or building icon).
218. Click it and select **Course Sandbox** (your course environment from Lab 0).

Note: ⚠ **Warning:** Copilot Studio **must use the same environment as Power Automate**. If your agent is built in one environment and your flows live in another, they will not be able to connect to each other in Day 3. Always confirm the environment name in the top-right before you start. The environment also needs **Dataverse** enabled, because agents are stored there.

Step 2: Create a blank agent and configure it (~10 minutes)

219. On the **Home** page you will see a large box inviting you to **describe** the agent you want in natural language. You could create the agent by chatting — but for this lab you will configure everything yourself so you understand every field.

220. In the left navigation, select **Agents**, then select **Create blank agent**. *(On the **Home** page the same option appears as **Create an agent** under **Start building from scratch**.)*
221. Wait a few seconds while the agent is provisioned. Its **Overview** page then opens.
222. In the **Details** section of the Overview page, select **Edit** and fill in: — **Name:** `Company Helpdesk` — **Description:** `Answers general questions about ACME Pte Ltd for staff and customers.`

Select **Save**.

223. In the **Instructions** section of the Overview page, select **Edit**, copy and paste the text below into the **Instructions** box, then select **Save**:

```
    You are the Company Helpdesk agent for ACME Pte Ltd.
    Be friendly, concise, and professional.
    Answer questions about our products, opening hours, and contact details.
    If you do not know an answer, politely say so and suggest emailing
help@acme.example.
    Keep replies to 2-3 sentences.
```

Note: **Tip:** Think of the three fields like this — **Name** is what people see, **Description** is a short note for *you* (and helps other agents/tools recognise it later), and **Instructions** are the actual rules the AI follows in every conversation. Instructions are where almost all of your effort goes.

Step 3: Tour the agent workspace (~5 minutes)

Now that the agent exists, get familiar with the layout. Near the top of the page you will see several tabs. The exact labels can shift slightly between versions, but you will find these building blocks:

224. **Overview** — a summary of your agent and quick links to its parts.
225. **Knowledge** — the documents, websites, and data the agent can read to find facts (you will use this in Step 4 and much more in Lab 7).
226. **Topics** — scripted, step-by-step conversation flows you design by hand for predictable questions.
227. **Tools** (called **Actions** in older versions) — things the agent can *do*, such as send an email or run a Power Automate flow (covered in Lab 8).
228. The **Test** pane — a chat panel, usually on the right. If you do not see it, select **Test** at the top-right.

Click through each tab once so you know where things are. You do not need to change anything yet.

Note: **Tip:** Remember the four building blocks: **Instructions** = behaviour and tone, **Knowledge** = facts the agent can look up, **Topics** = scripted conversations, **Tools** = things the agent can do. Almost everything in Copilot Studio is one of these four.

Step 4: Add one Knowledge source (~10 minutes)

Right now your agent only knows what is in its Instructions. Give it something real to answer from.

229. Open the **Knowledge** tab.
230. Select **+ Add knowledge**.
231. Choose **Public website** (the simplest option — no files needed).
232. In the URL box, paste a relevant website. For practice, use the Microsoft Copilot Studio docs: — `https://learn.microsoft.com/microsoft-copilot-studio/` — *(Alternatively, if you have a short company FAQ as a PDF or Word file, choose the **Files** option and upload it instead.)*
233. Give the source a short, clear **name** such as `Copilot Studio docs` and, if asked, a one-line **description** of what it contains.
234. Select **Add to agent**.
235. Watch the source's status. It will show **Processing** (or a spinner) and then change to **Ready**. Wait until it says **Ready** before testing — this can take a minute or two.

Note: **Tip:** A website source lets the agent ground its answers in real content instead of guessing. You will go much deeper on Knowledge in Lab 7.

Step 5: Test your agent (~7 minutes)

236. Open the **Test** pane on the right. If it is hidden, select **Test** at the top-right.
237. Type these questions one at a time and read the replies: — `What can you help me with?` — `What is Copilot Studio?` — `How do I contact support?`
238. Notice two things working together: — Your **Instructions** control the *style* — friendly, short (2-3 sentences), professional. — Your **Knowledge** controls the *facts* — answers drawn from the website you added.

Note: **Tip:** The Test pane updates as you build. After most changes you make, come back here and re-test. This build-then-test loop is exactly how real agents are developed.

Step 6: Iterate on your Instructions (~5 minutes)

The first version of an agent is rarely perfect. Tuning the Instructions is normal and expected.

239. Suppose replies are too long or too formal. Open the **Overview** tab and select **Edit** in the **Instructions** section.
240. Add a line to make the change you want, for example:

```
Always greet the user by name if they give it.
Never give legal or financial advice; suggest contacting a manager instead.
```

241. Select **Save**.
242. Go back to the **Test** pane. If your earlier conversation is still showing, select the **Start new test session** icon (the circular-arrow / refresh icon) at the top of the Test pane so it picks up the new Instructions, then ask again.

Note: ⚠ **Warning:** The Test pane does **not** always pick up changes automatically. If your edit does not seem to take effect, select the **Start new test session** (refresh) icon at the top of the Test pane to restart the conversation.

Step 7: Publish the agent (optional) (~5 minutes)

Publishing makes your latest version live so it can be shared. You do not have to deploy it anywhere yet.

Note: **Note:** A Copilot Studio **trial** license lets you create and test agents in the Test pane, but does **not** let you publish them. If Publish is blocked on your account, simply read through this step — everything else in the course works from the Test pane.

243. Select **Publish** (top-right), then confirm by selecting **Publish** again in the dialog.
244. Once publishing finishes, open the **Channels** tab. **Channels** are the places people can use your agent — for example Microsoft Teams, a demo website, or a custom app.
245. Browse the list to see your options. You do not need to connect any channel now; publishing alone is enough to keep your latest version testable.

Checkpoint

You have successfully completed this lab when:

- ✅ You confirmed you are in the **Course Sandbox** environment (matching Power Automate)
- ✅ An agent named **Company Helpdesk** exists with your custom **Instructions**
- ✅ At least one **Knowledge** source shows the status **Ready**
- ✅ The agent gives sensible, on-style answers in the **Test** pane
- ✅ You edited the Instructions and saw the change after refreshing the Test pane

Troubleshooting

Problem	Cause	Solution
Agent ignores your new instructions	Test pane is showing the old conversation	Select **Save**, then select the **Start new test session** (refresh) icon at the top of the **Test** pane to restart the chat.
Knowledge source stuck on "Processing"	Large website or busy service	Wait a minute or two. If it persists, remove it and add a smaller single page or a short PDF instead.
Replies are generic or	No grounding, or vague	Make sure a Knowledge source

unhelpful	instructions	is **Ready** and tighten your Instructions (be specific about tone and scope).
Can't see the Test pane	Pane is collapsed	Select **Test** at the top-right of the agent page.
Tools/flows won't connect later	Wrong environment	Use the **environment selector** (top-right) to switch to the same environment as Power Automate (**Course Sandbox**).
Can't find **Create blank agent**	UI variation / older environment	On the **Home** page use **Create an agent** under **Start building from scratch**. If your tenant still shows the older "Describe your agent" screen, select **Skip to configure** (top-right of that screen) instead, then edit the fields on the Overview tab.

Key Takeaways

- An agent's behaviour is shaped mainly by its **Instructions** — clear instructions give predictable answers.
- The four building blocks are **Instructions** (behaviour/tone), **Knowledge** (facts), **Topics** (scripted chats), and **Tools** (doing things).
- **Knowledge** grounds answers in real content so the agent stops guessing.
- The **Test** pane is your build-and-iterate loop — refresh it after each change.
- Always work in the **same environment** as your Power Automate flows, and that environment needs **Dataverse**.

Duration

~45 minutes

Next Steps

Proceed to Lab 7: Add Knowledge to Your Agent.

Lab 7: Add Knowledge to Your Agent (Grounding / RAG)

Lab Title

Ground Your Agent in Your Own Documents with Knowledge (RAG)

Lab Objectives

By the end of this lab, you will be able to:

246. Explain **grounding** and **RAG** (Retrieval-Augmented Generation) in plain, everyday terms
247. Add one or more **Knowledge** sources to a Copilot Studio agent (files, public website, or SharePoint)

248. Turn off **Allow ungrounded responses** (general knowledge) so the agent answers only from your content
249. Test that the agent answers **from your documents** and shows **citations**
250. Confirm the agent declines to answer when the information is not in your sources (no made-up answers)
251. Tell the difference between **Instructions** (behaviour) and **Knowledge** (facts)

Prerequisites

- Completed Lab 6 (you have an agent)
- A short document to use as knowledge (a PDF/Word FAQ, or a public website URL)

Scenario

Your **Company Helpdesk** agent from Lab 6 can chat, but it does not actually know **ACME Pte Ltd's** real facts — its return policy, its opening hours, its product list. An agent that only knows general internet information is not very useful for *your* business.

In this lab you will give the agent a **Knowledge** source containing ACME's own information, then prove that the agent answers from that source and nothing else. This is the technique that makes business agents trustworthy.

Note: **Tip: Grounding** means tying the agent's answers to specific source material you provide, so it cannot just invent things. **RAG** (Retrieval-Augmented Generation) is the technology that does this.

How RAG works (in plain terms)

252. You add documents to **Knowledge**. Copilot Studio breaks them into small passages and indexes them.
253. A user asks a question.
254. The agent **retrieves** the few passages most relevant to that question.
255. The agent **generates** an answer using *only* those passages, and shows **citations** pointing back to the source.

Note: **RAG in one line:** *Find the relevant text in your documents → hand it to the AI → it answers from that text, with citations.* This keeps answers accurate and up to date without retraining any model.

Step-by-Step Guide

Step 1: Prepare a knowledge source (~5 minutes)

Pick **one** source to start (you can add more later). Choose whichever is easiest for you:

256. **A file (recommended for this lab):** Create a one-page document called `ACME Company FAQ`. Save it as a **PDF** or **Word** file. Include a few clear facts the agent will answer from, for example:

```
ACME Pte Ltd - Company FAQ

Opening hours: Monday to Friday, 9am to 6pm. Closed on public holidays.
Return policy: Items may be returned within 30 days with a receipt.
Contact: Email help@acme.example or call +65 6123 4567.
Top products: ACME Widget Pro, ACME Gadget Lite, ACME Toolkit.
```

257. **A public website:** any URL whose content you want the agent to use (e.g. a real product page).
258. **A SharePoint** site or document library, if your tenant has one with relevant files.

Note: **Tip:** Keep the document focused and tidy — use clear headings and short paragraphs. Clean, well-structured content leads to much better retrieval and more accurate answers.

Step 2: Open your agent's Knowledge tab (~3 minutes)

259. Go to **https://copilotstudio.microsoft.com** and confirm the **environment selector** (top-right) shows **Course Sandbox**.
260. Open your **Company Helpdesk** agent from Lab 6.
261. Select the **Knowledge** tab.
262. Select **+ Add knowledge**.

Note: ⚠ **Warning:** Make sure you are in the **same environment** as your Lab 6 agent and your Power Automate flows. If the environment is wrong, you may be editing a different (or empty) agent.

Step 3: Add the knowledge source (~10 minutes)

263. On the **Add knowledge** screen, choose the type that matches your source: — **Files** / **Upload** — then select and upload your `ACME Company FAQ` PDF or Word file. — **Public website** — then paste the URL. — **SharePoint** — then paste the site or library URL.
264. Give the source a clear **name**, for example `ACME Company FAQ`.
265. If you are prompted for a **description**, write a short sentence describing what it contains, e.g. `ACME opening hours, return policy, contact details, and top products.` The agent uses this description to decide when this source is relevant.
266. Select **Add to agent**.
267. Watch the source's **status**. It will show **Processing** and then change to **Ready**. Larger files and websites take a little longer. (A single uploaded file can be up to **512 MB**, and an agent can hold up to **500 files** — your one-page FAQ is nowhere near the limits.)

Note: ⚠ **Warning:** Do **not** test until the status reads **Ready**. While a source is still **Processing**, the agent cannot use it and may reply that it has no information.

Step 4: Turn off general knowledge (optional but recommended) (~4 minutes)

By default the agent may blend in general AI/web knowledge. For a business helpdesk you usually want answers **only** from ACME's documents.

268. Open the agent's **Settings** (top-right) and select the **Generative AI** page.
269. In the **Knowledge** section, find **Allow ungrounded responses** (this is the current name of the old *"Use general knowledge"* toggle — it controls whether the agent may answer from the AI model's own general knowledge without using your sources).
270. Turn it **Off** so the agent relies only on your knowledge sources and tools.
271. Also check the **Use information from the web** setting is **Off** (it also appears as the **Web Search** toggle in the **Knowledge** section of the agent's **Overview** page), so answers don't come from a live Bing web search either.
272. Select **Save**.

Note: **Tip:** Turn **Allow ungrounded responses** (and **Web Search**) **on** when you want the agent to also answer broad, everyday questions. Turn them **off** when you need tight control and want to prevent answers that did not come from your documents.

Step 5: Test that answers come from your document (~10 minutes)

273. Open the **Test** pane and select the **Start new test session** (refresh) icon at the top so it picks up your new Knowledge.
274. Ask questions that can **only** be answered from your `ACME Company FAQ`, for example: — `What is your return policy?` — `What are your opening hours?` — `Which products do you offer?`
275. Confirm each answer matches the facts in your document.
276. Look **underneath each answer** for **citations** or **references** (often numbered, like `[1]`, or a "1 reference" link). Click a citation to confirm it points back to your source.

Note: **Tip:** Citations are your proof that grounding worked. If an answer has no citation, it may have come from general knowledge rather than your document.

Step 6: Confirm the agent declines when it doesn't know (~5 minutes)

This is the most important test — it proves the agent will not make things up.

277. In the **Test** pane, ask something that is **not** in your document, for example: — `What is your CEO's home address?` — `Do you sell aeroplanes?`
278. The agent should say it does not have that information (and, following your Lab 6 Instructions, suggest emailing `help@acme.example`).

279. If instead it confidently invents an answer, that is a **hallucination**. Re-check that **Allow ungrounded responses is Off** (Step 4) and that your Instructions tell it to answer only from provided sources.

Note: **Tip:** A trustworthy business agent saying *"I don't have that information"* is a success, not a failure. Declining gracefully is exactly the behaviour you want.

Step 7: Tune your sources (~5 minutes)

280. If an answer is weak or missing, improve the **source document** — add clearer headings and more detail — then re-upload or refresh the source.
281. Add a **second** Knowledge source for broader coverage (for example, a product page website). Give each source a **distinct name and description** so the agent can tell them apart.
282. Reinforce grounding in the **Instructions** (Overview tab). Add a line such as:

```
Only answer using the provided knowledge sources, and cite them.
If the answer is not in the sources, say you don't have that information.
```

283. Select **Save**, refresh the Test pane, and re-test.

Checkpoint

You have successfully completed this lab when:

- ✅ At least one **Knowledge** source shows the status **Ready**
- ✅ The agent answers business questions **from your document**, with visible **citations**
- ✅ **Allow ungrounded responses** is turned **Off** (if you chose the recommended setup)
- ✅ The agent **declines** to answer (no hallucination) when the information is not in your sources
- ✅ You can explain the difference between **Instructions** and **Knowledge**

Troubleshooting

Problem	Cause	Solution
Source stuck on "Processing"	Large or busy source	Wait a minute or two; if it persists, try a smaller file or a single web page.
Agent ignores the document	Source not ready, or no description	Confirm the source is **Ready**; add a clear description; in Instructions, tell it to answer from knowledge.
Answers come from the web, not my doc	Web search / ungrounded answers are on	Turn **off Allow ungrounded responses** and **Use information from the web** in Settings → **Generative AI** (Knowledge section), then

		Save and refresh the Test pane.
No citations appear	Answer was not grounded	Ensure the answer actually came from a Ready source; check that **Allow ungrounded responses** is off.
Agent makes things up	Hallucination	Turn off **Allow ungrounded responses** and add an Instruction to decline when info is missing.
Edits not reflected in chat	Test pane caching	Select the **Start new test session** (refresh) icon at the top of the **Test** pane to restart the conversation.

Key Takeaways

- **Knowledge = grounding (RAG):** the agent retrieves passages from your documents and answers from them, with citations.
- Use **Instructions** for *behaviour and tone*; use **Knowledge** for *facts*.
- Turning **Allow ungrounded responses off** (and **Web Search off**) keeps answers strictly inside your own content.
- **Citations** let you and your users verify exactly where an answer came from.
- A grounded agent that **declines** when it doesn't know is working correctly — that is how you avoid hallucinations.

Duration

~40 minutes

Next Steps

Proceed to Lab 8: Add Tools and Actions to Your Agent.

Lab 8: Add Tools and Actions to Your Agent

Lab Title

Give Your Agent Tools — Connectors, Prebuilt Actions, and Flows

Lab Objectives

By the end of this lab, you will be able to:

284. Explain what **Tools/Actions** are and how they differ from **Knowledge**
285. Add a **prebuilt connector action** as a tool (e.g. Send an email V2) and create its connection
286. Write a clear tool **name and description** that tells the agent *when* to use the tool
287. Set tool **inputs**, either as fixed values or filled by AI from the conversation

288. (Alternative) Add a **multi-step Power Automate agent flow** as a tool (using **Add a row into a table**), and **disable a competing tool** so the agent calls the right one
289. Test the agent actually **performing an action**, and read the **Activity map**

Prerequisites

- Completed Lab 6 and Lab 7
- A working **Office 365 Outlook** connection (a mailbox-enabled account you can sign in with)

Scenario

So far your **Company Helpdesk** agent can *answer* questions (using Knowledge). But staff at **ACME Pte Ltd** also want it to *do* things — for example, email the support team when someone needs help escalated.

Knowledge lets an agent answer; Tools let it act. In this lab you will give your agent a tool so a chat conversation can trigger real work, such as sending an email. This is the bridge to the end-to-end agent-plus-flow workflows you will build in Day 3.

Note: **Knowledge vs Tools:** - **Knowledge** = read and answer from your documents (Lab 7). - **Tools/Actions** = perform actions in other systems — send an email, create a record, run a Power Automate flow (this lab).

Step-by-Step Guide

Step 1: Open the Tools tab (~5 minutes)

290. Go to **https://copilotstudio.microsoft.com** and confirm the **environment selector** (top-right) shows **Course Sandbox** — the same environment as your Power Automate flows.
291. Open your **Company Helpdesk** agent from Lab 6.
292. Select the **Tools** tab (this was labelled **Actions** in older versions).
293. Select **+ Add a tool**. Microsoft groups tools into several **core tool types**: — **Prebuilt / custom connector action** — a single ready-made operation from a connector like Office 365 Outlook, Excel, or Teams (you use this in Steps 2–4). — **Agent flow** — a multi-step Power Automate flow with conditions and logic, used as one tool (you use this in Step 5). — **Prompt** — a single-turn AI prompt that returns text. — **REST API / MCP tool / Computer use** — connect to web services, a Model Context Protocol server, or GUI automation (advanced; out of scope here).

Note: **How the agent picks a tool:** With **generative orchestration**, the agent chooses *which* tool to call at runtime **based purely on each tool's name and description** — there are no fixed trigger phrases. That is why a clear, intent-rich **description** (Step 3) is

the single most important thing you write, and why two tools with overlapping descriptions confuse the agent (see Step 5a).

Note: ⚠ **Warning:** Tools and the flows they call must live in the **same environment** as the agent. If your flow was built in a different environment, the agent will not be able to see or call it.

Step 2: Add a prebuilt connector action (~10 minutes)

You will add a "send an email" action directly as a tool.

294. Select **+ Add a tool**, then apply the **Connector** filter.
295. In the search box, type `Send an email` and select **Send an email (V2)** under **Office 365 Outlook**.
296. If you are prompted to **sign in / create a connection**, do so now using a **mailbox-enabled account** (an account that can actually send email). Approve any consent prompt.
297. The action is added to your tools list. Open it to see its **inputs**: **To**, **Subject**, and **Body**.

Note: ⚠ **Warning:** If you ever see an **Unauthorized** error when the tool runs, the connection is broken or signed in with the wrong account. Open the tool's **connection** settings and **reconnect** with a mailbox-enabled account.

Step 3: Name and describe the tool (~5 minutes)

The description is the most important part — it is how the agent decides **when** to call this tool.

298. Open the tool and set a clear **name** and **description**, for example: — **Name:** `Send notification email` — **Description:** `Use this to email a summary to the support team when the user asks to escalate an issue or notify someone.`
299. Be specific. A vague description like `sends email` makes the agent unsure when to use it; a precise description makes it reliable.

Note: **Tip:** The agent reads the tool **description** the way you would read a label. If the label clearly says what the tool is for, the agent picks it at the right moment.

Step 4: Configure the tool's inputs (~7 minutes)

Decide how each input (To, Subject, Body) gets its value. You have two choices per input:

300. **Fixed value** — you type the value once and it never changes. Good for **To**: set it to your own email so test messages go to you, e.g. `you@yourcompany.com`.

301. **Filled by AI** (dynamic) — the agent fills the value from the conversation. Choose **Fill using AI** (or **Dynamically**) for **Subject** and **Body** so the agent writes them based on what the user said.

Note: **Tip:** For your first test, set **To** as a **fixed** value (your own inbox). That way you can confirm the email actually arrives without risking sending it to the wrong person.

Step 5: (Alternative path) Add a Power Automate flow as a tool (~15 minutes)

A single connector action (Steps 2–4) is perfect for **one** step. But real business work is usually **multi-step** — you often need to *log a record* **and** *notify someone* **and** *return a result* in one go. A single connector tool cannot do that; a **Power Automate agent flow** can. In this step you will build a flow that **logs a support request to a table**, then add that flow to the agent as a second tool.

Note: **Why a different action here?** In Steps 2–4 your agent already learned to *send an email* with a single connector tool. To show what a flow adds, this step uses a **different** action — **Add a row into a table** — so the flow *records* the request instead of emailing it. (Do **not** add another *Send an email* action here; that would just duplicate the tool you already built and confuse the agent about which one to call.)

Step 5a — Turn OFF the connector tool from Step 4 first (do not skip)

If you leave the **Send notification email** connector tool (Steps 2–4) **enabled** while you add and test the new flow tool, your agent now has **two** tools that both look like "do something with the user's request." The orchestrator may keep choosing the old email tool and never call your new flow — making it look like the flow is broken when it is not. Disable the old tool so this exercise has exactly **one** active tool to reason about.

302. In your agent, open the **Tools** tab.
303. Find the **Send notification email** tool you added in Steps 2–4.
304. Select its **… (More actions)** menu and turn the tool **Off** (toggle/disable it). *Do not delete it* — you will switch it back on later.
305. Confirm the tool now shows as **Off / Disabled** in the list before continuing.

Note: ⚠ **Warning:** This was the step most learners discovered was missing. With **both** tools on, the agent often fires the Step 4 email tool instead of the new flow, and the flow appears to "do nothing." Turning the connector tool **Off** isolates the flow so you can confirm it works.

Step 5b — Create the agent flow

306. In Copilot Studio's left navigation, select **Flows**, then **New flow → Agent flow**. *(From inside the agent you can instead use the **Tools** tab → **+ Add a tool → New Agent flow**.)*
307. The **agent flow designer** opens (the same designer Power Automate uses) with two steps already in place: the trigger **When an agent calls the flow** and a

Respond to the agent action. Every agent flow follows this three-part shape: **trigger** → **do the work** → **respond**.

Step 5c — Define the flow's input

308. Select the trigger **When an agent calls the flow**, then **+ Add an input**.
309. Choose **Text**, name it `Request summary`, and give it the hint `A short summary of the support request`. This is the value the agent will pass in from the conversation.
310. Open the **Overview** tab → **Details** → **Edit**, set **Flow name** to `Log support request` and **Description** to `Logs a support request as a new row in the requests table`, then **Save**.

Step 5d — Add the work: "Add a row into a table" (the only action here)

311. Back on the **Designer** tab, select the **+** between the trigger and **Respond to the agent** to insert an action.
312. Search for `Excel`, choose **Excel Online (Business)**, then the action **Add a row into a table** (reuse the workbook and table from Lab 2, or any table with a `Request` column).
313. Map the row's **Request** column to the trigger's `Request summary` input using **Dynamic content**.

Note: **Reminder:** Add **a row into a table** here — **not** a *Send an email* action. The email path was already covered by the connector tool in Steps 2–4.

Step 5e — Respond to the agent

314. Select the **Respond to the agent** node, then **+ Add an output**.
315. Choose **Text**, name it `status`, and set its value to `Logged` (or use **Dynamic content** to return the new row's ID). The flow **must** end here, or the agent never gets a result back.

Step 5f — Publish

316. Select **Save draft**, then **Publish**.
317. Return to **Tools** and confirm the flow's status is **Ready**.

Step 5g — Add the flow to the agent and map its input

318. In the agent, open the **Tools** tab → **+ Add a tool**, apply the **Flow** filter, select **Log support request**, then **Add and configure**.
319. Set a clear **Description**: `Records a support request to the requests table when the user asks to log or file an issue.`
320. Under **Additional details**, review **When this tool may be used**, **Ask the end user before running**, and **Credentials to use** (end-user credentials, with a sign-in prompt like `Please sign in to log the request`).

321. Under **Inputs**, for the `Request summary` input set **Fill using** → **Dynamically fill with AI** so the agent writes the summary from the conversation (or **Custom value** to bind a variable), just like the fixed-vs-AI choice in Step 4.
322. Under **Completion**, set **After running** → **Write the response with generative AI** so the agent confirms the result in natural language. **Save**.

Note: **Tip:** Every flow used as a tool follows the same three-part shape: **When an agent calls the flow** (trigger) → **do the work** → **Respond to the agent** (give a result back). You will reuse this pattern throughout Day 3.

Step 6: Test the agent performing the action (~8 minutes)

323. Open the **Test** pane and select the **Start new test session** (refresh) icon at the top so it picks up the new flow tool.
324. Type a message that should trigger the flow, for example: — `Please log a support request: I need help with order 123.`
325. Watch the agent decide to **call the Log support request flow**. If it asks for confirmation or shows a sign-in prompt, approve it.
326. Confirm the action really happened — open your table and check that a **new row** was added with the request summary.
327. Open the **Activity map** (a diagram of the conversation, if shown) to see the tool being called and the exact data passed into it. This is the best way to understand *why* the agent did what it did.
328. **Switch the email tool back on:** return to the **Tools** tab and turn the **Send notification email** tool (Steps 2–4) back **On**. With both tools live and clearly described, the agent can now *log* a request **and** *email* support — choosing each by its **description**.

Note: **Tip:** If the flow does nothing, check three things in order: is the **Send notification email** tool still **Off** during the isolation test, did the flow **Publish** to status **Ready**, and did the agent actually choose the flow (look at the **Activity map**)?

Step 7: Refine the tool (~5 minutes)

329. **Agent never calls the tool?** Improve the tool **description** to clearly state the trigger situations, and add an example trigger phrase in the agent's **Instructions** (e.g. *"When a user asks to notify or escalate, use the Send notification email tool."*).
330. **Agent calls the tool with the wrong data?** Tighten the input setup — use a fixed value where the data should never change, or add a clearer hint for the AI-filled inputs.
331. **Save**, refresh the Test pane, and test again.

Checkpoint

You have successfully completed this lab when:

- ✅ A **tool** (a connector action and/or a flow) is added to your agent
- ✅ The tool has a clear **name and description** that tells the agent when to use it
- ✅ The tool's **connection** is signed in with a mailbox-enabled account (no Unauthorized error)
- ✅ The agent **performs the action** during a test conversation (e.g. the email arrives)
- ✅ You found the tool call in the **Activity map**

Troubleshooting

Problem	Cause	Solution
Agent never calls the tool	Description too vague	Rewrite the **description** to name the exact situations; add a trigger example in **Instructions**.
Connection / Unauthorized error	Broken connection or wrong account	Open the tool's connection and **reconnect** with a **mailbox-enabled** account.
Email never arrives	Wrong recipient or tool not called	Check the **To** value, confirm the connection, and verify the tool ran in the **Activity map**.
Wrong data sent to the tool	AI filled inputs incorrectly	Use **fixed values** where data shouldn't change, or add clearer input hints; test again.
Tool not listed after adding	Wrong environment, or not refreshed	Confirm the agent and flow are in the **same environment** (Course Sandbox); refresh the page.
Flow tool fails to return	No response step	Make sure the flow ends with **Respond to the agent** with an output, then re-publish it.
Agent calls the old email tool, never the flow	Two competing tools both enabled	Turn the **Send notification email** connector tool **Off** while you test the flow (Step 5a), then test again.
Flow shows up but status isn't **Ready**	Not published	**Save draft** then **Publish** the flow; the **Tools** list must show status **Ready** before the agent can call it.

Key Takeaways

- **Tools/Actions** let an agent *act* — using connectors, prebuilt actions, or full Power Automate flows.
- The tool **description** is what makes the agent call the right tool at the right time — write it carefully.
- Inputs can be **fixed** or **filled by AI** from the conversation; use fixed values for anything that must not change.
- A flow used as a tool always goes **When an agent calls the flow → do the work → Respond to the agent**.
- Use a **flow** (not a single connector action) when the work is **multi-step** — e.g. **Add a row into a table** to log a record and return a result.

- When two tools overlap, the agent may pick the wrong one — **turn off** the competing tool while you test, then re-enable it once each tool has a precise description.
- A broken connection causes an **Unauthorized** error — reconnect with a mailbox-enabled account.
- Flows-as-tools are the foundation of the end-to-end workflows you build in Day 3.

Duration

~45 minutes

Next Steps

Proceed to Lab 9: Sales Enquiry Assistant.

Lab 9: Sales Enquiry Assistant

Lab Title

Build a Sales Enquiry Assistant that Captures Structured Data

Lab Objectives

By the end of this lab, you will be able to:

332. Create a Copilot Studio **agent** from scratch in the correct environment
333. Build a **topic** and set its **trigger** so the agent knows when to run it — a **description** for generative orchestration (default), or **trigger phrases** for classic orchestration
334. Add **Ask a question** nodes that save answers into named **variables**
335. Choose the right **Identify** type so text and numbers are captured cleanly
336. Return a tidy **structured summary** that inserts every captured variable
337. Understand how these variables will later feed a Power Automate flow (Lab 10 and Day 3)

Prerequisites

- Completed Lab 6
- Read Module 3, especially *Prompt design for structured outputs*

Note: **Tip:** Sign in to Copilot Studio at https://copilotstudio.microsoft.com and confirm the environment shown at the **top-right** says **Course Sandbox** before you start. Every lab on Day 2 must be built in this same environment, otherwise your agent and your flow will not be able to see each other later.

Scenario

You work at **ACME Pte Ltd**. The sales team complains that enquiries arrive in inconsistent formats — some by email, some by phone notes — so details get lost. Your job is to build a **Sales Enquiry Assistant** agent that walks a customer through a short set of questions and captures **name, company, product, and quantity** as separate, named pieces of data. The result is a clean **structured summary** — exactly the kind of tidy data a Power Automate flow can log and route automatically in the next lab.

Step-by-Step Guide

Step 1: Create the agent (~5 minutes)

338. In the left navigation, select **Agents**, then select **Create blank agent**. *(On the **Home** page the same option appears as **Create an agent** under **Start building from scratch**.)* Wait a few seconds while the agent is provisioned; its **Overview** page opens.
339. In the **Details** section of the Overview page, select **Edit** and fill in, then **Save**: — **Name:** `Sales Enquiry Assistant` — **Description:** `Captures sales enquiries (name, company, product, quantity) as structured data for ACME Pte Ltd.`
340. In the **Instructions** section, select **Edit**, paste the text below, then **Save**:

```
    You are a Sales Enquiry Assistant for ACME Pte Ltd.
    Your job is to collect a customer's enquiry details: full name, company,
    product of interest, and quantity.
    Collect one item at a time and be polite and concise.
    Do not answer unrelated questions; gently steer back to capturing the
enquiry.
```

Note: **Tip:** The **Description** matters more than it looks: with generative orchestration, the agent (and any parent agent) uses names and descriptions to decide when things are relevant. Keep it specific.

Step 2: Create the "New Sales Enquiry" topic (~10 minutes)

Note: ⚠ **Read this first — the latest Copilot Studio works differently.** New agents now use **generative orchestration** by default. In this mode a topic is triggered by its **name and description** — the agent *chooses* the topic when the user's message matches that description — **not** by a fixed list of "Phrases". The old phrase list still exists, but it's now called **User says a phrase** and only appears if you deliberately switch the trigger type. This lab uses the modern **description-based** trigger (recommended and easiest); the phrase option is shown at the end of this step.

341. Open your **Sales Enquiry Assistant** agent, then in the agent's top menu select **Topics**. *(If you don't see it, select ⋯ **More** and choose **Topics**.)*
342. Select **+ Add a topic**, then choose **From blank**. A blank canvas opens with a single **Trigger** node at the top. *(You may also see **Create from description with Copilot** — ignore it for this lab so you build the steps yourself.)*

343. On the canvas toolbar, select **Details** to open the **Topic details** panel, then set: — **Name:** `New Sales Enquiry` *(don't use a full stop/period in a topic name)* — **Description:** `Use this topic when the customer wants to make a sales enquiry, ask for a quote, or say they are interested in buying a product. It collects the customer's name, company, product, and quantity.`

Close the panel. This **Description** is what makes the agent pick this topic, so keep it specific.

344. Select the **Trigger** node once. For a generative-orchestration agent it reads **The agent chooses** — that is correct, and it uses the description from step 3. **You do not need to add any phrases.**
345. Select **Save** (top-right of the canvas).

Note: **Tip:** The agent matches on *meaning*, not exact wording. A clear, specific **Description** is what makes the topic fire reliably — write it the way you'd explain to a colleague *when* this topic should be used.

(Optional) Want exact trigger phrases instead? Only if your agent uses **classic orchestration**, or you specifically want the topic to fire on set phrases:

346. Hover over the **Trigger** node and select the **Change trigger** icon.
347. Choose **User says a phrase**.
348. Add example phrases (aim for 5–10), each on its own line: `I want to make an enquiry`, `I'd like a quote`, `new sales enquiry`, `I'm interested in a product`.
349. Select **Save**.

Step 3: Capture the customer's name and company (~10 minutes)

350. Under the **Trigger** node, select the **Add node** icon (**+**), then choose **Ask a question**. A **Question** node appears.
351. Configure the question node: — **Ask a question (message):** `Sure, I can help with that! What is your full name?` — **Identify:** open this dropdown and choose **User's entire response**. This captures the whole reply as plain text (good for names). — **Save user response as:** Copilot Studio creates a variable automatically. Select the variable name and rename it to `customerName`.
352. Select the **Add node** icon (**+**) below that node and add a second **Ask a question** node for the company: — **Ask a question (message):** `Thanks! Which company are you with?` — **Identify: User's entire response** — **Save user response as:** rename the variable to `company`

Note: **Tip:** Rename variables right away. A clear name like `customerName` is much easier to find later than the default `Var1`, especially when you map it to a flow in Lab 10.

Step 4: Capture the product and quantity (~10 minutes)

353. Add another **Ask a question** node for the product: — **Ask a question (message):** `Which product are you interested in?` — **Identify: User's entire response** — **Save user response as:** `product`
354. Add a final **Ask a question** node for the quantity: — **Ask a question (message):** `How many units would you like?` — **Identify:** open the dropdown and choose **Number**. This forces the answer to be a real number, so it can be used in calculations and stored as a number later. — **Save user response as:** `quantity`

Note: ⚠ **Warning:** Make sure quantity uses **Identify = Number**, not "User's entire response". If you leave it as text, `quantity` becomes a word like "twenty-five" instead of `25`, and the Excel/flow steps in Lab 10 will store messy data.

Note: **Tip:** Capturing into four separate named variables (`customerName`, `company`, `product`, `quantity`) is what makes the output **structured** — each piece of data is isolated and reusable. A single blob of free text could not be logged into separate spreadsheet columns.

Step 5: Confirm and summarize (~10 minutes)

355. Select the **Add node** icon (**+**) below the quantity question and choose **Send a message**. A **Message** node appears.
356. Type the summary text below. Wherever you see a curly-brace variable, do **not** type it as plain text — instead place your cursor there, select the **{x}** (Insert variable) button on the node, and pick the matching variable from the list:

```
Thank you! Here is a summary of your enquiry:
• Name: {customerName}
• Company: {company}
• Product: {product}
• Quantity: {quantity}
A sales representative from ACME will contact you within 1 business day.
```

357. Select **Save**.

Note: ⚠ **Warning:** If you type `{customerName}` as literal text, the customer will literally see the words "{customerName}". You must insert it through the **{x}** button so it shows up as a coloured variable token.

Step 6: Test the assistant (~10 minutes)

358. Open the **Test** pane on the right (if it was already open, select the **Start new test session** refresh icon so it picks up your latest changes).
359. Type a message that starts an enquiry, e.g. `I'd like a quote`. The agent should recognise this from your topic **Description** and start the **New Sales Enquiry** topic.
360. Answer each question in turn: — Name: `Mei Ling` — Company: `BrightTech` — Product: `Air Fryer Pro` — Quantity: `25`

361. Confirm the agent returns the structured summary with all four values filled in correctly.
362. Now test the **unhappy path**: start the topic again, and when asked for quantity, type the word `twenty-five` instead of a number. Because **Identify** is set to **Number**, the agent should **re-ask** the question. Type `25` and confirm it then accepts it.

Note: **Tip:** The re-ask behaviour is a feature, not a bug. It guarantees `quantity` is always a clean number before it reaches a flow.

Checkpoint

You are ready to move on when all of the following are true:

- ✅ An agent named **Sales Enquiry Assistant** exists in the **Course Sandbox** environment
- ✅ It has a **New Sales Enquiry** topic with a clear **trigger** — a description (**The agent chooses**) or trigger phrases
- ✅ Four variables are captured: `customerName`, `company`, `product`, and `quantity` (Number)
- ✅ A structured summary with all four values is returned during testing
- ✅ Typing text for the quantity causes the agent to re-ask

Troubleshooting

Problem	Solution
I can't find a "Phrases" trigger type	New agents use **generative orchestration**, so the trigger is **The agent chooses** (description-based). Either write a clear topic **Description**, or hover the **Trigger** node → **Change trigger** → **User says a phrase** to add phrases.
Topic doesn't trigger when I type	Make the topic **Description** more specific about *when* to use it (or add more varied phrases if using **User says a phrase**). Type something close in meaning; re-test after **Save**.
Variable shows as blank in the summary	You probably typed the variable name as text. Delete it and re-insert it using the **{x}** button. Also confirm the **Save user response as** name matches.
Summary shows literal `{customerName}` text	Same cause — insert variables via **{x}**, do not type curly braces by hand.
Quantity answer is rejected or re-asked forever	Set the quantity question's **Identify** to **Number** and answer with digits like `25`, not spelled-out words.
Test pane shows old behaviour	Select the **Start new test session** (refresh) icon at the top of the Test pane to reload the latest version of the topic.
Agent answers off-topic questions	Tighten the **Instructions** to say it must steer back to capturing the enquiry.

Key Takeaways

- **Topics** are triggered by a **description** (**The agent chooses**, generative orchestration — the default) or by **trigger phrases** (**User says a phrase**, classic orchestration).
- **Ask a question** nodes capture answers into named **variables**, which is the foundation of structured data.
- The **Identify** type matters: use **User's entire response** for free text and **Number** for numeric answers.
- A confirmation summary builds user trust and lets you visually verify the captured data before any automation runs.
- Clean, separated variables here are what make the agent-to-flow integration in Lab 10 possible.

Duration

~45 minutes

Next Steps

Proceed to Lab 10: Procurement Request Workflow, where you'll connect a capturing agent to a Power Automate flow for the first time.

Lab 10: Procurement Request Workflow

Lab Title

Connect a Procurement Agent to a Power Automate Flow

Lab Objectives

By the end of this lab, you will be able to:

363. Prepare an Excel **table** so a flow can log structured rows into it
364. Build an agent topic that captures a procurement request as named **variables**
365. Create an **agent flow** in Power Automate triggered **when an agent calls the flow**
366. Define flow **inputs** and map agent variables into them
367. Have the flow **log the request to Excel** and **email the procurement team** in one run
368. Return a confirmation from the flow back to the agent with **Respond to the agent**

Prerequisites

- Completed Lab 9 (capturing variables)
- Completed Day 1 Labs 1–2 (email + Excel actions)

Note: ⚠ **Warning:** Your agent (Copilot Studio) and your flow (Power Automate) must live in the **same environment — Course Sandbox**. If they are in different environments, the agent will not be able to find or call its flow. Before you begin, open both Copilot Studio and Power Automate and confirm the **top-right environment picker** shows **Course Sandbox** in each.

Scenario

At **ACME Pte Ltd**, staff currently email the procurement team to request supplies, and someone manually copies each request into a spreadsheet. You'll replace that with an agent. When a staff member finishes their request, the agent **automatically logs** it to a shared spreadsheet **and emails** the procurement team — no manual copying, no missed requests. This is your first true **agent + flow** integration: the agent gathers the data, and a Power Automate flow does the real work.

Step-by-Step Guide

Step 1: Prepare the procurement log in Excel (~5 minutes)

369. In Excel (saved to **OneDrive for Business**, not your local PC), create a new workbook and name it `Procurement Log`.
370. In **row 1**, type these six headers, one per cell from A1 to F1: `Date`, `Requester`, `Item`, `Quantity`, `Reason`, `Status`.
371. Select the range **A1:F1**, then go to the **Insert** tab and select **Table**. In the dialog, tick **My table has headers** and select **OK**.
372. With the table selected, open the **Table Design** tab. In the **Table Name** box (far left), replace the default name with `ProcurementTable`. Press **Enter**.
373. Save and close the workbook.

Note: ⚠ **Warning:** The file must be in **OneDrive for Business** (or SharePoint), not on your local hard drive. A flow's Excel actions can only see files stored in the cloud. The exact table name `ProcurementTable` matters — the flow will look for it by name.

Step 2: Create the agent and capture the request (~10 minutes)

374. In Copilot Studio, select **Agents** in the left navigation, then **Create blank agent** (as in Lab 9). When the new agent's **Overview** page opens, select **Edit** in the **Details** section, fill in the fields below, and **Save**; then select **Edit** in the **Instructions** section, paste the instructions, and **Save**: — **Name:** `Procurement Assistant` — **Description:** `Captures staff purchase requests for ACME Pte Ltd and submits them for processing.` — **Instructions:**

```
You are a Procurement Assistant for ACME Pte Ltd.
Collect a staff member's purchase request: requester name, item, quantity,
and reason. Collect one item at a time, be concise, then confirm the
```

```
request
     has been submitted.
```

375. Open the **Topics** tab, select **+ Add a topic**, then **From blank**. On the toolbar select **Details** and set the **Name** to `New Procurement Request`.
376. Give the topic a clear **trigger** so the agent knows when to run it: — **Latest Copilot Studio (generative orchestration — default):** the **Trigger** node reads **The agent chooses**. In the **Details** panel, set the **Description** to `Use this topic when a staff member wants to raise a procurement or purchase request to buy supplies. It collects their name, item, quantity, and reason.` No phrases are needed. — **(Optional) Classic orchestration / exact phrases:** hover the **Trigger** node → **Change trigger** → **User says a phrase**, then add phrases such as `procurement request`, `I need to buy something`, `I want to order supplies`, `raise a purchase request`.
377. Add four **Ask a question** nodes in order, using the **Add node** icon (**+**) each time. For each, set the **Identify** type and rename the **Save user response as** variable exactly as shown: — Question `What is your name?` → Identify **User's entire response** → save as `requester` — Question `What item do you need?` → Identify **User's entire response** → save as `item` — Question `How many do you need?` → Identify **Number** → save as `quantity` — Question `What is the reason for this request?` → Identify **User's entire response** → save as `reason`
378. Select **Save**.

Note: ⚠ **Warning:** Set the quantity question's **Identify** to **Number**. If it stays as text, the Excel **Quantity** column will store words instead of numbers.

Step 3: Create the agent flow (~15 minutes)

379. Still in the **New Procurement Request** topic, select the **+** node after the last question, choose **Add a tool**, then **New Agent flow**. — *(Alternatively, from the agent's **Tools** tab, select **Add a tool**, then **New agent flow**.)*
380. The **agent flow designer** opens (the same designer Power Automate uses) with the trigger **When an agent calls the flow** and a **Respond to the agent** action already in place.
381. Confirm you are still working in the **Course Sandbox** environment (top-right).
382. On the **When an agent calls the flow** trigger, select **+ Add an input** and create four inputs. Choose the matching type and name each one exactly: — Type **Text**, name `requester` — Type **Text**, name `item` — Type **Number**, name `quantity` — Type **Text**, name `reason`

Note: **Tip:** Keeping flow input names identical to your topic variable names (`requester`, `item`, `quantity`, `reason`) makes the mapping step later much less confusing.

Step 4: Add the flow actions — log to Excel and email (~15 minutes)

Action 1 — Add a row into a table (Excel Online Business):

383. Select the **+** between the trigger and **Respond to the agent**, search for **Add a row into a table**, and select it (Excel Online (Business)).
384. Set the file location fields: — **Location:** OneDrive for Business — **Document Library:** OneDrive — **File:** browse to and select `Procurement Log.xlsx` — **Table:** select `ProcurementTable` from the dropdown
385. Map the columns. For the **Date** column, you must enter an expression — do not type the date by hand: — Click the **Date** field, then select the **fx** (Expression / function) tab in the dynamic content panel. — Type this expression into the fx editor, then select **Add / OK**:

```
formatDateTime(utcNow(),'yyyy-MM-dd HH:mm')
```

- The field should now show a blue/purple **token**, not plain text.

386. Map the remaining columns using the **Dynamic content** tab — pick the values that come from the **trigger** (the inputs you defined in Step 3): — **Requester:** `requester` — **Item:** `item` — **Quantity:** `quantity` — **Reason:** `reason` — **Status:** type the literal text `Pending`

Note: ⚠ **Warning:** The `formatDateTime(utcNow(),'yyyy-MM-dd HH:mm')` expression must be entered through the **fx editor** so it becomes a token. If you paste it as plain text into the Date field, Excel will store the literal characters `formatDateTime(...)` instead of a real date.

Action 2 — Send an email (Office 365 Outlook):

387. Select the **+** below the Excel action (still above **Respond to the agent**), search for **Send an email (V2)** (Office 365 Outlook), and select it.
388. Fill in the email: — **To:** the procurement team address (use your own email for testing) — **Subject:** type `New Procurement Request from` , then insert the **requester** dynamic content right after it — **Body:** type the text below, inserting each value from the **Dynamic content** panel where shown (do not type the variable names by hand):

```
A new procurement request has been submitted:
Requester: {requester}
Item: {item}
Quantity: {quantity}
Reason: {reason}
Status: Pending
```

Action 3 — Respond to the agent:

389. Select the **Respond to the agent** action that was added for you at the end of the flow.

390. Select **+ Add an output**, choose **Text**, name it `result`, and set its value to: `Logged and procurement team notified.`
391. Select **Save draft**, then **Publish** — the agent can only call a **published** flow.
392. Return to your agent in Copilot Studio (select **Go back to agent** if prompted).

Note: ⚠ **Warning:** If the **Send an email** action shows an **"Unauthorized"** error, the Office 365 Outlook **connection** is broken or signed in with an account that has no mailbox. Open the action's **…** menu, select **My connections**, and **reconnect** using a mailbox-enabled work account. Every connection used by the flow must show a green ✓ before the flow will run.

Step 5: Wire the flow into the topic (~10 minutes)

393. Back in Copilot Studio, the flow should now appear as a tool/action node inside the **New Procurement Request** topic. If it does not appear, **refresh** the page and make sure the flow was **published** in the **same environment (Course Sandbox)**.
394. Select the tool node. Under its **inputs**, map each input to the matching topic variable — this step is mandatory; the flow receives nothing if you skip it: — input `requester` → variable `requester` — input `item` → variable `item` — input `quantity` → variable `quantity` — input `reason` → variable `reason`
395. After the tool node, add a **Send a message** node that shows the value the flow returned. Insert the `result` output via the **{x}** button:

```
{result} Your request has been recorded. Thank you!
```

396. Select **Save** to save the topic.

Note: ⚠ **Warning:** Mapping is the step everyone forgets. After adding the flow as a tool, you **must** map each flow input to its matching topic variable. Unmapped inputs arrive empty, so your Excel row and email will be blank.

Step 6: Test end-to-end (~10 minutes)

397. Open the **Test** pane and select the **Start new test session** (refresh) icon so it loads the latest topic.
398. Type a message that starts the request, e.g. `procurement request` — the agent recognises it from the topic **Description** (or phrases). Answer the questions: — Name: `Daniel` — Item: `Wireless mouse` — Quantity: `10` — Reason: `New hires`
399. Confirm all three results occurred: — The agent shows the confirmation message ("Logged and procurement team notified. Your request has been recorded. Thank you!"). — A **new row** appears in **Procurement Log.xlsx** (with a real date, the four values, and Status `Pending`). — The **notification email** arrives in your inbox.
400. If anything is missing, open the flow in Power Automate and review its **run history** (28-day run history) — select the latest run to inspect the inputs each step received and any error messages.

Checkpoint

You are ready to move on when all of the following are true:

- ✅ A **Procurement Log** workbook with a **ProcurementTable** exists in OneDrive
- ✅ A **Procurement Assistant** agent captures four variables (`requester`, `item`, `quantity`, `reason`)
- ✅ An **agent flow** triggered by **When an agent calls the flow**, with four matching inputs
- ✅ One conversation produces both a **new Excel row** and a **notification email**
- ✅ The agent shows the flow's returned confirmation message

Troubleshooting

Problem	Solution
Flow doesn't appear in the topic	Refresh the Copilot Studio page; confirm the flow was **published** and is in the **same environment (Course Sandbox)**.
Inputs unmapped or arriving empty	Select the tool node and map **each** flow input to its matching topic variable.
"Unauthorized" error on Send an email	Reconnect the **Office 365 Outlook** connection with a mailbox-enabled work account; every connection must show a green ✓.
Date column shows `formatDateTime(...)` as text	You typed the expression instead of entering it in the **fx editor**. Re-enter it via **fx** so it becomes a token.
Excel row is blank	Map the columns to the **trigger** dynamic content (the inputs), not to outputs of later steps.
No email received	Check your Junk folder, verify the **To** address, and review the flow **run history** for errors.
Flow can't find the table	Confirm the file is in **OneDrive for Business** and the table is named exactly `ProcurementTable`.

Key Takeaways

- An **agent flow** uses the **When an agent calls the flow** trigger and ends with **Respond to the agent**.
- Agent **variables** map to flow **inputs**; this mapping is mandatory and easy to forget.
- The flow can **return** a value (`result`) that the agent shows back to the user.
- Dates come from the `formatDateTime(utcNow(),'yyyy-MM-dd HH:mm')` expression entered via the **fx editor** as a token — never typed as text.
- One short conversation now triggers real business actions: **logging** and **notifying** together.

Duration

~50 minutes

Next Steps

Proceed to Lab 11: Automated Response Generation.

Lab 11: Automated Response Generation

Lab Title

Use AI Prompts to Generate Professional Responses Automatically

Lab Objectives

By the end of this lab, you will be able to:

401. Add an AI Builder **Run a prompt** action inside a flow
402. Feed captured enquiry data into the prompt to generate tailored text
403. Control the **length, tone, and format** of the AI output
404. Email the AI-generated response automatically
405. Compare a static template versus an AI-generated response and know when to use each

Prerequisites

- Completed Lab 10
- A **Sales Enquiry Assistant** agent (from Lab 9) you can reuse

Note: ⚠ **Warning:** As in Lab 10, the agent and the flow must both be in the **Course Sandbox** environment, or the agent cannot call its flow. Confirm the environment picker (top-right) in both Copilot Studio and Power Automate.

Scenario

At **ACME Pte Ltd**, every sales enquiry currently gets the same copy-paste reply. It works, but it feels impersonal. In this lab you'll reuse your **Sales Enquiry Assistant** from Lab 9 and add a flow that uses an **AI prompt** to draft a warm, personalised reply from the customer's captured details — then emails it automatically. This shows how generative AI turns the structured data you captured into polished, individual communication, while still keeping you in control of length and tone.

Step-by-Step Guide

Step 1: Understand where the AI prompt runs (~5 minutes)

406. An AI prompt can run in two equivalent places: — **In a Power Automate flow** — using the **Run a prompt** action (AI Builder). — **In Copilot Studio** — by adding a **Prompt** tool to a topic.
407. This lab uses the **flow** approach because it builds directly on the agent + flow skills from Lab 10 and keeps the "generate" and "email" steps together in one place.

Note: **Tip:** Both options call the same underlying AI Builder model. Choosing the flow approach simply means generation and emailing live in one flow you can inspect via run history.

Step 2: Reuse the Sales Enquiry Assistant and add a flow (~10 minutes)

408. Open your **Sales Enquiry Assistant** agent (from Lab 9) and open the **New Sales Enquiry** topic.
409. After the summary **Send a message** node, select the **+** node, choose **Add a tool**, then **New Agent flow**.
410. The **agent flow designer** opens with the trigger **When an agent calls the flow** and a **Respond to the agent** action already in place. Confirm you are still in the **Course Sandbox** environment.
411. On the trigger, select **+ Add an input** and add four inputs (matching the variables from Lab 9): — Type **Text**, name `customerName` — Type **Text**, name `company` — Type **Text**, name `product` — Type **Number**, name `quantity`

Step 3: Add the AI prompt action (~15 minutes)

412. Select the **+** between the trigger and **Respond to the agent**, search for **Run a prompt** under **AI Builder**, and select it. *(This action was called "Create text with GPT using a prompt" until May 2025; the separate "Create text with GPT" action is deprecated.)*
413. In the action's **Prompt** dropdown, choose **+ New custom prompt** (rather than one of the prebuilt prompts). The **prompt builder** opens.
414. Write the prompt instruction below. Wherever a value comes from the enquiry, insert it from the **Dynamic content** panel rather than typing it:

```
   Write a warm, professional email reply (maximum 120 words) to a sales
enquiry
   for ACME Pte Ltd.
   Customer name: {customerName}
   Company: {company}
   Product of interest: {product}
   Quantity: {quantity}

   The email should:
   - Thank the customer by name
   - Acknowledge the product and quantity
   - Say a sales representative will follow up within 1 business day with
pricing
   - End with a professional sign-off from "The ACME Sales Team"
   Output only the email body text. Do not include the prompt or these
instructions.
```

415. Select **Save** on the prompt. The action now produces a generated **Text** output.

Note: **Tip:** Notice the prompt explicitly states **length** (max 120 words), **tone** (warm, professional), **content** (what to mention), and **format** ("output only the email body").

Controlling all four is what separates a reliable AI step from an unpredictable one — this is the structured prompt design from Module 3 applied to output generation.

Step 4: Email the generated response (~10 minutes)

416. Select the **+** below the prompt action (still above **Respond to the agent**), search for **Send an email (V2)** (Office 365 Outlook), and select it.
417. Fill in the email: — **To:** your own email (for testing; in production you would use a captured customer email variable) — **Subject:** type `Re: Your enquiry about` , then insert the **product** dynamic content — **Body:** insert the prompt's **Text** output (the generated email text) from the **Dynamic content** panel
418. Select the **Respond to the agent** action at the end of the flow, select **+ Add an output**, and return a **Text** output named `result` with value `Reply drafted and sent.`
419. Select **Save draft**, then **Publish** the flow, then return to your agent in Copilot Studio.

Note: ⚠ **Warning:** If **Send an email** shows an **"Unauthorized"** error, reconnect the **Office 365 Outlook** connection with a mailbox-enabled work account. Every connection used by the flow must show a green ✓ before it will run.

Step 5: Wire the inputs and test (~10 minutes)

420. Back in the topic, select the flow's tool node and map each input to its matching topic variable — the flow gets empty values if you skip this: — input `customerName` → variable `customerName` — input `company` → variable `company` — input `product` → variable `product` — input `quantity` → variable `quantity`
421. (Optional) After the tool node, add a **Send a message** node showing `{result}` so the agent confirms the email was sent.
422. Select **Save** to save the topic.
423. Open the **Test** pane, select the **Start new test session** (refresh) icon, type a message that starts the enquiry (the agent recognises it from the topic **Description**), and run the enquiry: — Name: `Mei Ling`, Company: `BrightTech`, Product: `Air Fryer Pro`, Quantity: `25`
424. Confirm: — The flow runs successfully (check **run history** if unsure). — You receive a **uniquely worded, personalised** email — not a fixed template.
425. Run it again with different details and compare the two emails; the AI adapts the wording each time.

Note: **Tip:** If the email body contains the prompt instructions, your "Output only the email body text" line is missing or you mapped the wrong field — map only the prompt **output**, and keep that instruction in the prompt.

Step 6: Compare template versus AI (~5 minutes)

426. Recall the **static template** email from Day 1: it is fast and 100% predictable, but identical for every customer.
427. The **AI-generated** response is personalised and natural, but you must guide it with a precise prompt and review it for accuracy.
428. Note the best-practice rule: use AI for **drafting the wording**, and keep critical facts (prices, policies, legal terms) as controlled static text or knowledge — never let the model invent them.

Note: **Tip:** A safe production pattern is "AI drafts, human (or fixed text) confirms the facts." Personalisation from AI plus controlled facts gives you the best of both.

Checkpoint

You are ready to finish when all of the following are true:

- ✅ A flow that calls an **AI prompt** with the four enquiry inputs
- ✅ A personalised email generated and sent automatically from the agent conversation
- ✅ The email contains **only the body** (no leaked prompt instructions)
- ✅ You can explain when to use a static template versus an AI-generated reply

Troubleshooting

Problem	Solution
No **Run a prompt** action available	Search for **Run a prompt** under **AI Builder**; confirm AI Builder is available in this environment (the trial / developer environment includes it). The old **Create text with GPT** action is deprecated — do not use it.
"Unauthorized" error on Send an email	Reconnect the **Office 365 Outlook** connection with a mailbox-enabled account; every connection must show a green ✓.
Output too long or wrong tone	Tighten the prompt: state the word limit, the tone, and "output only the email body."
Captured values not appearing in the email	Confirm you inserted them as **dynamic content** inside the prompt, and that the topic inputs are **mapped** to the variables.
Email body shows the prompt instructions	Add "Output only the email body text" to the prompt and map only the prompt **output** into the email body.
Flow doesn't appear in the topic	Refresh Copilot Studio; ensure the flow is **published** in the **same environment (Course Sandbox)**.

Key Takeaways

- **AI prompts** turn the structured data you captured into polished, personalised text.
- You control quality by specifying **length, tone, content, and format** — especially "output only the email body."
- Combine AI drafting with controlled facts for responses that are both personal and reliable.

- The same agent + flow pattern from Lab 10 now powers generative output, not just logging.

Duration

~45 minutes

Next Steps

You've completed Day 2. Proceed to Day 3 — Module 4: End-to-End Orchestration Concepts.

Day 3 — End-to-End Workflow Automation & Workshop

Module 4: End-to-End Workflow Automation

Note: Read this before the Day 3 labs. ~15 minutes.

You now have both halves of an automated business process. Today you join them into complete, end-to-end workflows — and learn the design discipline that keeps them reliable.

---

1. Agents + Power Automate = end-to-end automation

By now you've built both halves:

- **Power Automate flows** (Day 1) — the *doing*: send email, log to Excel, run approvals.
- **Copilot Studio agents** (Day 2) — the *understanding*: capture requests as structured data and call flows.

End-to-end automation chains these into a complete business process where a **single trigger flows all the way through to a finished outcome** — with little or no manual work in between:

```
CAPTURE  →  RECORD  →  DECIDE  →  NOTIFY  →  CLOSE
(agent/    (Excel    (approval  (email/    (final
 email/     log)      or rule)   Teams)     status)
 form)
```

One event goes in the top; a finished, recorded, communicated result comes out the bottom.

---

2. Workflow orchestration

Orchestration is arranging multiple steps — and sometimes multiple flows and agents — so they run in the right order, pass data correctly, and handle every outcome.

A well-orchestrated workflow has five qualities:

429. **A clear trigger** — one starting event (email arrives, file uploaded, agent finishes capturing).
430. **Sequenced actions** — each step's **output** feeds the next step's **input**.
431. **Branching** — conditions route the process (approved vs rejected, high value vs low value).
432. **Notifications** — the right people are told at the right moments.
433. **A definite end state** — the record is updated to its final status (Logged / Approved / Rejected / Completed).

Note: **No dead ends.** Orchestration is mostly about making sure *every* path — including the unhappy ones — leads somewhere defined. A rejected request still needs an owner, a notification, and a final status.

Common orchestration patterns

You'll build one of each this morning. Notice how they grow in complexity:

Pattern	Shape	What's new	Day 3 lab
Capture → Log → Notify	Linear: record it, tell someone	Pure sequence, no branch	Lab 12
Upload → Approve → Act	File trigger, then human decision	Adds an approval + branch	Lab 13
Request → Approve → Notify	Multi-party approval chain	Adds routing by who/what	Lab 14

Note: **Approver reminder (from Day 1):** the approver in any approval step must be a real **user in your tenant**, not an outside email address. Pick a colleague — or yourself — for testing.

---

3. Managing workflow outputs and next steps

Every step produces **outputs**. Managing them well is what separates a demo from a dependable workflow:

- **Pass data forward** with dynamic content — sender's email → notification; captured quantity → approval title. (Use the **fx** expression editor for any small formatting, like a date or a trimmed string.)
- **Use a Status column** as the source of truth: `New → Pending → Approved/Rejected → Done`. Update it at each stage so anyone can see where a request stands at a glance.
- **Decide the next step for every outcome.** Never leave a branch empty — an approved request and a rejected request *both* need a defined follow-up.

- **Notify deliberately.** Tell the requester the result; tell the team when action is needed. Avoid noisy, redundant emails.
- **Make it traceable.** The Excel log **plus** the Power Automate **run history** together give you a complete audit trail.

Designing a workflow — a simple method

Use this every time *before* you start clicking in the designer:

434. **Write the sentence:** *"When ___ happens, do ___, then ___, and finally ___."*
435. **Identify the trigger** (the "when").
436. **List the actions** in order (the "do / then / finally").
437. **Mark the decision points** (where it branches).
438. **Define each end state** (final status + who is notified).
439. **Build, test the happy path, then test every branch.**

Note: **Sentence first, build second.** If you can't say the workflow in one plain sentence, you're not ready to build it yet. The sentence reveals your trigger, your steps, and your end states before you touch the tool.

You'll apply exactly this method in **Lab 16 (Capstone Workshop)**.

---

4. What you'll build on Day 3

- **Lab 12:** Email enquiry → log to Excel → notify the team (Capture → Log → Notify).
- **Lab 13:** Invoice file uploaded → approval workflow (Upload → Approve → Act).
- **Lab 14:** Purchase request → manager approval → notification (Request → Approve → Notify).
- **Lab 15:** Order Processing — an agent captures an order, then the flow confirms, logs, and raises a restock alert.
- **Lab 16:** Capstone — design and build your **own** end-to-end workflow for Sales, Finance, Procurement, or Order Processing.

Labs 12–14 drill the three core patterns; Lab 15 brings the agent back into the loop; Lab 16 is where you put it all together yourself.

Next: Lab 12: Email Enquiry → Excel Logging → Notification

Module 5: Business Workflow Workshop

Note: **Read this before the Capstone (Lab 16).** ~10 minutes.

This is your briefing for the capstone. Everything you've learned across three days now comes together in one workflow that you design, build, and explain.

1. Purpose of the workshop

You've now learned every building block:

- **Power Automate** flows — email, Excel logging, approvals, scheduled and event triggers (Day 1)
- **Copilot Studio** agents — instructions, knowledge/RAG, tools, structured capture (Day 2)
- **End-to-end orchestration** — combining agents + flows across business domains (Day 3 morning)

The workshop is where you **put it together yourself**: design and build a complete, working workflow for a real business process — and explain its value to the business.

Note: Think of this as moving from *following* recipes to *cooking your own dish* with the techniques you've practised.

2. The four business domains

Your capstone targets one (or more) of these domains. Each maps directly to labs you've already built — so you're never starting from a blank page; you're adapting and combining work you understand.

Domain	Typical workflow	Built on
Sales	Enquiry/lead capture → log → acknowledge → notify rep	Labs 9, 12
Finance	Invoice/expense submitted → approve by amount → notify	Labs 3, 13
Procurement	Purchase request → manager approval → notify + log	Labs 10, 14
Order Processing	Order captured → confirm → log → restock alert	Lab 15

Note: **Pick the domain you understand best.** A workflow you can explain confidently is worth more than an ambitious one you can't finish.

3. Design method (recap from Module 4)

Run through this **before** you build — it's the same discipline from Module 4:

440. **One sentence:** *"When ___ happens, do ___, then ___, and finally ___."*
441. **Trigger:** manual / scheduled / form / email / file / agent call
442. **Capture:** the fields/variables you need
443. **Actions in order:** log, generate, approve, notify…
444. **Decision points:** the condition(s) and each branch
445. **End states:** final status + who is notified for each outcome

If you can write step 1 cleanly, the rest of the design tends to fall into place.

---

4. Quality bar

A strong capstone workflow:

- ✅ Has a clear **trigger** and runs **end to end**
- ✅ **Captures or receives** data and **logs** it (Excel)
- ✅ Includes at least one **condition** (branching)
- ✅ Includes an **approval** *or* an **AI-generated** response
- ✅ **Notifies** the right people for **every** outcome
- ✅ Is **tested** on the happy path **and** every branch

Note: Treat this as your self-check list. Before you call the capstone "done," walk down it and tick each item.

---

5. Tips for success

- **Start small, then add.** Get a basic happy-path flow working end to end first, then layer on conditions, approvals, and notifications.
- **Test after each step** — don't build everything and then test once. Small tests catch problems while they're still easy to find.
- **Reuse your earlier labs as templates** — see the Capstone's reuse list. Adapting working flows is faster and safer than rebuilding from scratch.
- **Check connections are green before running** — a red connector or an **Unauthorized** error means: reconnect first.
- **Remember the approver rule** — approvals must go to a real **user in your tenant** (yourself is fine for testing), never an outside personal email.
- **Use a Status column** in Excel as the single source of truth, updated at each stage.
- **Keep agent output structured** — clean variables in means clean flow inputs out.

Note: **One sentence, built carefully, tested fully.** That's a winning capstone. Good luck.

Next: Lab 16: Capstone Workshop

Lab 12: Email Enquiry → Excel Logging → Notification

Lab Title

End-to-End Workflow: Capture an Email Enquiry, Log It, and Notify the Team

Lab Objectives

By the end of this lab, you will be able to:

446. Create an **automated cloud flow** that starts on its own when an email arrives
447. Configure the **When a new email arrives (V3)** trigger with an optional subject filter
448. **Log** each enquiry as a new row in an Excel table, with a clean timestamp
449. **Notify** the sales team automatically with the enquiry details
450. Orchestrate the classic **Capture** → **Log** → **Notify** pattern into one hands-off workflow

Prerequisites

- Completed Day 1 Labs 1–2 (email + Excel actions)
- An Excel `Enquiry Log` with table `EnquiryTable` (from Lab 2) — or create one (Date, Name, Email, Message, Status)

Scenario

ACME Pte Ltd receives customer enquiries by email to a shared inbox. Right now, staff copy each enquiry into a spreadsheet by hand and forget to tell the sales team. In this lab you will build an automated flow in the **Course Sandbox** environment that, the moment an enquiry email arrives, **logs it to Excel** and **emails the sales team** — completely hands-off. This is the classic **Capture** → **Log** → **Notify** pattern.

Step-by-Step Guide

Step 1: Create an automated cloud flow (~5 minutes)

451. In your browser, go to **https://make.powerautomate.com** and sign in.
452. Top-right, confirm the environment selector shows **Course Sandbox**. If it shows a different environment, click it and switch — every tool in this course uses the same environment.
453. In the left menu, select **Create**.
454. Under **Start from blank**, select **Automated cloud flow**.
455. In the **Build an automated cloud flow** dialog: — **Flow name:** `Lab 12 - Email Enquiry to Excel and Notify` — In **Choose your flow's trigger**, type `when a new email arrives` in the search box. — Select **When a new email arrives (V3)** under **Office 365 Outlook**.
456. Select **Create**.

Note: **Tip:** If this is your first Office 365 / Excel action, Power Automate may ask you to sign in to create a **connection**. Always sign in with your own course account. A working connection shows a green ✓ — you will rely on this later.

Step 2: Configure the email trigger (~10 minutes)

457. The flow opens in the new designer with the trigger card already placed. Select the **When a new email arrives (V3)** card to open its panel on the right.
458. Set (if a field is not visible, open the **Advanced parameters** dropdown and select **Show all**): — **Folder:** `Inbox` (use the folder picker — do not type it) — **Importance:** `Any` — **Only with Attachments:** `No` — **Include Attachments:** `No`
459. Still under **Advanced parameters**, set: — **Subject Filter:** `Enquiry` — This means only emails whose subject contains the word **Enquiry** will start the flow — which keeps your log clean while testing.

Note: **Tip:** Without a subject filter, **every** incoming email starts the flow. During training, set a filter so unrelated mail (newsletters, replies) does not flood your Excel log.

Step 3: Log the enquiry to Excel (~12 minutes)

460. Under the trigger card, select the **+** (plus) → **Add an action**.
461. Search `add a row into a table` and select **Add a row into a table** under **Excel Online (Business)**.
462. In the action panel set: — **Location:** `OneDrive for Business` — **Document Library:** `OneDrive` — **File:** browse to and select **Enquiry Log.xlsx** — **Table:** select **EnquiryTable** from the dropdown
463. The five table columns (Date, Name, Email, Message, Status) now appear. Map them: — **Name:** click the field, then from **Dynamic content** insert **From** (or **From** display name if you prefer) — **Email:** insert **Dynamic content** → **From** — **Message:** insert **Dynamic content** → **Body preview** — **Status:** type `New`
464. For the **Date** column, enter an expression so the timestamp is clean: — Click the **Date** field, then open the **fx** (Expression) editor. — Type exactly: `formatDateTime(utcNow(),'yyyy-MM-dd HH:mm')` — Select **Add** (or **OK**). The field now shows a blue **fx token**, not the raw text.

Note: ⚠ **Warning:** Enter the date through the **fx** editor so it becomes a token. If you type `formatDateTime(...)` directly into the box as plain text, Excel stores the literal text and you get a meaningless string instead of a date.

Step 4: Notify the sales team (~10 minutes)

465. Under the Excel action, select **+** → **Add an action**.
466. Search `send an email` and select **Send an email (V2)** under **Office 365 Outlook**.
467. Configure: — **To:** the sales team address — for testing, use your own email — **Subject:** type `New enquiry received:` then insert **Dynamic content** → **Subject** — **Body:** type the message below, inserting the matching **Dynamic content** tokens where shown in braces:

```
A new enquiry has arrived and been logged.
```

```
From: [From]
Subject: [Subject]
Message: [Body preview]

Status: New – please follow up.
```

468. Select **Save** (top right).

Note: ⚠️ **Warning:** If saving or testing later shows **"Unauthorized"** on the Send an email action, your Office 365 Outlook connection has expired or used the wrong account. Select the action and, at the bottom of its configuration pane (next to **Connected to …**), select **Change connection** and **reconnect** with a **mailbox-enabled** account. Every connection must show a green ✓ before the flow will run.

Step 5: Test the end-to-end workflow (~10 minutes)

469. Top right, select **Test** → **Manually** → **Test**.
470. From another account (or the same one), **send an email** to your inbox with: — **Subject:** `Enquiry - bulk order` — **Body:** a short sentence, e.g. `Hi, I'd like a quote for 200 units.`
471. Within about a minute the flow should trigger. Confirm: — Every step shows a **green ✓** in the run view. — A **new row** appears in **Enquiry Log.xlsx** with the sender, message, and a readable date/time. — The **notification email** arrives with the enquiry details.
472. Now send a **second** test email **without** the word "Enquiry" in the subject (e.g. subject `Hello`). Confirm the flow does **not** run — proving the subject filter works.

Note: **Tip:** If the new Excel row shows `########` in the Date column, the column is just too narrow. Double-click the column border in Excel to auto-fit — the value is correct.

Step 6: Review and harden (~3 minutes)

473. In the left menu, open **My flows** → **Lab 12 - Email Enquiry to Excel and Notify** → the latest run to inspect inputs and outputs of each step.
474. Discuss optional improvements: — Add a **Condition** to flag high-priority enquiries (e.g. subject contains "urgent"). — Capture the sender's display name in a separate column.

Checkpoint

- ✅ Automated flow triggered by an **incoming email**
- ✅ Each matching email → **new Excel row** with a clean timestamp
- ✅ Each matching email → **team notification email**
- ✅ Non-matching emails are correctly **ignored** by the subject filter

Troubleshooting

Problem	Solution
Flow doesn't trigger	Confirm the email reached the **Inbox** and the subject contains `Enquiry`; triggers can take up to a minute.
"Unauthorized" on Send an email	Reconnect the **Office 365 Outlook** connection with a mailbox-enabled account; every connection must be green ✓.
Date shows raw text like `formatDateTime(...)`	You typed it as text. Re-enter it via the **fx** editor so it becomes a blue token.
Date column shows `########`	The column is too narrow — double-click the column border in Excel to auto-fit.
Empty Excel fields	Map to the **trigger's** dynamic content (From / Subject / Body preview), not static text.
Too many rows logged	Add or tighten the **Subject Filter** on the trigger.
No notification email	Check **Junk**; verify the **To** address; check run history for errors.

Key Takeaways

- Automated triggers (an **email arrives**) make workflows run with zero clicks.
- One trigger can fan out into multiple actions — **log** and **notify** in a single flow.
- Use the **fx** editor for dates so Excel stores a real, readable timestamp.
- Subject filters keep automation focused and your data clean.

Duration

~40 minutes

Next Steps

Proceed to Lab 13: Invoice Upload → Approval Workflow.

Lab 13: Invoice Upload → Approval Workflow

Lab Title

End-to-End Workflow: Trigger an Approval When an Invoice File Is Uploaded

Lab Objectives

By the end of this lab, you will be able to:

475. Use a **file upload** trigger — *When a file is created* in OneDrive
476. Pass file details into a **Start and wait for an approval** action
477. Assign the approval to a **real tenant user** so the run succeeds
478. **Branch** on the approval outcome with a Condition
479. **Move the file** and **notify** by outcome, completing the **Upload → Approve → Act** pattern

Prerequisites

- Completed Day 1 Lab 3 (approvals + conditions)
- Access to OneDrive (to create folders)

Scenario

At ACME Pte Ltd, staff drop supplier invoices into a shared OneDrive folder, but they sit unseen until someone remembers to check. In this lab you will build an automated flow in the **Course Sandbox** environment so that when an invoice lands in **Invoices/Incoming**, finance is asked to **approve or reject** it. Approved invoices move to an **Approved** folder and rejected ones to **Rejected**, with an email sent either way. This is the **Upload** → **Approve** → **Act** pattern.

Step-by-Step Guide

Step 1: Prepare the OneDrive folders (~5 minutes)

480. Go to **https://www.office.com**, sign in, and open **OneDrive**.
481. Select **+ Add new** → **Folder** and create a folder named `Invoices`.
482. Open the `Invoices` folder and create **three** subfolders inside it: — `Incoming` — `Approved` — `Rejected`
483. Leave `Incoming` empty for now — you will upload a test file later.

Note: **Tip:** Create all four folders **before** building the flow. The flow's trigger and **Move or rename a file** actions let you browse to these folders, which is far safer than typing paths by hand.

Step 2: Create the automated flow with a file trigger (~8 minutes)

484. Go to **https://make.powerautomate.com** and confirm the environment selector reads **Course Sandbox**.
485. In the left menu, select **Create** → **Automated cloud flow**.
486. In the dialog: — **Flow name:** `Lab 13 - Invoice Upload Approval` — Search `when a file is created` and select **When a file is created** under **OneDrive for Business**.
487. Select **Create**.
488. Select the trigger card to open its panel, then set: — **Folder:** use the folder picker (the small folder icon) and browse to `Invoices/Incoming`. Do not type the path.

Note: **Tip:** The trigger fires whenever a **new** file appears in that exact folder — that is your "upload" event. It does not fire for files already there before the flow was turned on.

Step 3: Start the approval (~10 minutes)

489. Under the trigger, select **+** → **Add an action**.

490. Search `start and wait for an approval` and select **Start and wait for an approval** under **Approvals**.
491. Configure: — **Approval type:** `Approve/Reject - First to respond` — **Title:** type `Invoice approval needed:` then insert **Dynamic content** → **File name** — **Assigned to:** start typing your **own name or course email**, then **pick the matching person from the people-picker dropdown** so it resolves to a **person chip** (a rounded tag with the name). — **Details:** type `Please review the uploaded invoice:` then insert **Dynamic content** → **File name**.

Note: ⚠ **Warning:** The **Assigned to** approver MUST be a user that already exists in **this** tenant. Pick the person from the dropdown so it becomes a **person chip** — do **not** type a free-text external email. If you paste an outside address, the run fails with *"The approvers/assigned to must contain valid users in the organization."*

Step 4: Branch on the outcome (~12 minutes)

492. Under the approval action, select **+** → **Add an action**, search `condition`, and select **Condition** (Control).
493. Configure the Condition: — Left value: insert **Dynamic content** → **Outcome** — Operator: `is equal to` — Right value: type `Approve` exactly (capital A, no quotes)

In the `True` branch (Approved):

494. Select **+** → **Add an action** → **Move or rename a file** (OneDrive for Business): — **File:** insert **Dynamic content** → **File identifier** (from the trigger) — **Destination File Path:** use the picker to select `Invoices/Approved`, then type `/` and insert **Dynamic content** → **File name** — the path must end with the file name — **Overwrite:** `Yes`
495. Select **+** → **Add an action** → **Send an email (V2)** (Office 365 Outlook): — **To:** your own email (for testing) — **Subject:** type `Invoice APPROVED:` then insert **Dynamic content** → **File name** — **Body:** `The invoice has been approved and moved to the Approved folder for payment processing.`

In the `False` branch (Rejected):

496. Select **+** → **Add an action** → **Move or rename a file**: — **File: Dynamic content** → **File identifier** — **Destination File Path:** pick `Invoices/Rejected`, then `/` + **File name** (as above) — **Overwrite:** `Yes`
497. Select **+** → **Add an action** → **Send an email (V2)**: — **To:** your own email — **Subject:** type `Invoice REJECTED:` then insert **Dynamic content** → **File name** — **Body:** `The invoice was rejected. Please review and resubmit if necessary.`
498. Select **Save**.

Note: ⚠️ **Warning:** Insert **File name** and **File identifier** as **single-value** dynamic content. If the designer ever wraps an action in an automatic **"Apply to each"** loop, you accidentally selected an array/list output. Delete the For each, then re-add the plain **Move or rename a file** / **Send an email** action and pick the single-value tokens (File name, File identifier). The Outcome and trigger file fields are all single-value.

Note: ⚠️ **Warning:** If **Send an email** shows **"Unauthorized"**, reconnect the **Office 365 Outlook** connection with a **mailbox-enabled** account. All connections must be green ✓ before the flow can run.

Step 5: Test both paths (~10 minutes)

499. Top right, select **Test** → **Manually** → **Test**.
500. In OneDrive, **upload a sample PDF** (any file) into `Invoices/Incoming`.
501. The flow triggers and pauses at the approval. Respond in **one** of these ways: — Open the **Approvals** hub from the left menu of make.powerautomate.com (look under **More** if it is not pinned), open the request, and choose **Approve**. — Or respond from the approval email / Teams card.
502. Confirm the **True** path: — The file **moved** from `Incoming` to `Invoices/Approved`. — You received the **APPROVED** email.
503. Repeat: upload **another** file, let the flow pause, and this time choose **Reject**. Confirm it moves to `Invoices/Rejected` and you get the **REJECTED** email.

Note: **Tip:** Approval emails can be slow or land in **Junk**. The most reliable place to respond is the **Approvals hub** in the left menu — it always shows pending requests assigned to you.

Step 6: Review (~5 minutes)

504. Open **My flows** → **Lab 13 - Invoice Upload Approval** → the latest runs and confirm the correct branch executed each time.
505. Optional enhancement: at the start of the flow, add a row to an `Invoice Register` Excel table with Status `Pending`, then update it after the decision for a full audit trail.

Checkpoint

- ✅ Flow triggered by a **file upload** to `Invoices/Incoming`
- ✅ Approval requested, **assigned to a real tenant user**, and awaited
- ✅ Approved → file in **Approved** folder + approval email
- ✅ Rejected → file in **Rejected** folder + rejection email

Troubleshooting

Problem	Solution
Run fails: "valid users in the organization"	**Assigned to** must be a real tenant user picked

	from the people-picker dropdown (a person chip) — not a typed external email.
Trigger doesn't fire	Confirm the file went into the exact `Invoices/Incoming` folder; allow up to a minute; the file must be **new**.
An "Apply to each" wrapped my action	You selected an array output. Delete the For each and re-add the plain action using single-value tokens (**Id**, **File name**, **Outcome**).
Move or rename a file fails	Use the trigger's **File identifier** as **File**; make sure the **Destination File Path** ends with the file name and the destination folders already exist.
Wrong branch runs	The Condition right value must equal exactly `Approve`.
"Unauthorized" on Send an email	Reconnect the **Office 365 Outlook** connection with a mailbox-enabled account; connections must be green ✓.
No approval email	Respond in the **Approvals hub**; check **Junk**; emails can be slow.

Key Takeaways

- A **file-created** trigger turns a simple upload into a full workflow.
- Approvals only run when **Assigned to** is a real tenant user chosen from the dropdown.
- Conditions route the file and notifications by the **Outcome**.
- Moving files between status folders gives a visible, auditable process.

Duration

~45 minutes

Next Steps

Proceed to Lab 14: Purchase Request → Manager Approval → Notification.

Lab 14: Purchase Request → Manager Approval → Notification

Lab Title

End-to-End Workflow: Agent-Captured Purchase Request with Manager Approval and Notification

Lab Objectives

By the end of this lab, you will be able to:

506. Combine an **agent** (capture) with an **agent flow** (log + approve + notify) in one process
507. Log a request to Excel as **Pending** with a clean timestamp
508. Use a **Condition** (cost threshold) to decide auto-approve vs manager approval

509. Assign approvals to a **real tenant user** and notify the requester of every outcome
510. Build a complete **Capture** → **Log** → **Approve** → **Notify** business workflow

Prerequisites

- Completed Day 2 Lab 8 (agent calling a flow) and Day 1 Lab 3 (approvals)
- An Excel `Procurement Log` with table `ProcurementTable` (from Lab 10) — add a `Status` column if missing

Scenario

At ACME Pte Ltd, staff raise purchase requests through the **Procurement Assistant** agent built in Day 2 Lab 10. In this lab you will extend it so each request is **logged to Excel**, routed by **cost threshold** (small requests auto-approve; larger ones go to a manager), and the requester is **automatically notified** of the result. Everything runs in the **Course Sandbox** environment — the full procurement loop, end to end.

Step-by-Step Guide

Step 1: Reuse the Procurement Assistant agent (~5 minutes)

511. Go to **https://copilotstudio.microsoft.com**, confirm the environment is **Course Sandbox**, and open your **Procurement Assistant** agent (from Day 2 Lab 10).
512. Open its **New Procurement Request** topic and confirm it captures these variables: `requester`, `requesterEmail`, `item`, `quantity`, `reason`. — If `requesterEmail` is missing, add an **Ask a question** node: `What is your email?` → save the answer to a variable named `requesterEmail`. — Add an `estCost` variable: add an **Ask a question** node `What is the estimated total cost (SGD)?`, set the **Identify** type to **Number**, and save to `estCost`. — Confirm `quantity` is also a **Number** variable.

Note: **Tip:** `quantity` and `estCost` must be **Number** variables, not Text. The Condition in Step 4 compares `estCost` to 500 numerically — a Text value can throw a comparison error.

Step 2: Build the agent flow (~8 minutes)

513. In the topic, after the questions, select **+** → **Add a tool** → **New agent flow**.
514. The flow opens with a **When an agent calls the flow** trigger. Select it and add these inputs (matching name and type exactly): — **Text** `requester` — **Text** `requesterEmail` — **Text** `item` — **Number** `quantity` — **Text** `reason` — **Number** `estCost`

Step 3: Log the request as Pending (~7 minutes)

515. Under the trigger, select **+** → **Add an action** → **Add a row into a table** (Excel Online (Business)).
516. Set **Location** `OneDrive for Business`, **Document Library** `OneDrive`, **File** `Procurement Log.xlsx`, **Table** `ProcurementTable`.
517. Map the columns from the flow's inputs (under **Dynamic content**): — **Requester:** `requester` — **Item:** `item` — **Quantity:** `quantity` — **Reason:** `reason` — **Status:** type `Pending` — (If your table has a Cost column, map `estCost`.)
518. For the **Date** column, click the field, open the **fx** (Expression) editor, type exactly `formatDateTime(utcNow(),'yyyy-MM-dd HH:mm')`, and select **Add**. The field shows a blue **fx token**.

Note: ⚠ **Warning:** Enter the date via the **fx** editor so it becomes a token. Typing the expression as plain text stores literal characters in Excel instead of a real timestamp. If Excel later shows `########` in the Date column, the column is simply too narrow — auto-fit it.

Step 4: Decide the approval path with a Condition (~12 minutes)

A common rule: small requests auto-approve; larger ones need a manager.

519. Under the Excel action, select **+** → **Add an action** → **Condition** (Control): — Left value: **Dynamic content** → **estCost** — Operator: `is greater than` — Right value: type `500`

In the `True` branch (needs manager approval):

520. Select **+** → **Add an action** → **Start and wait for an approval** (Approvals): — **Approval type:** `Approve/Reject - First to respond` — **Title:** type `Purchase approval:` then insert **item**, type `x`, then insert **quantity** — **Assigned to:** type your **own name or course email** and **pick the matching person from the people-picker dropdown** so it resolves to a **person chip**. — **Details:** `Requested by` + **requester** + `. Reason:` + **reason** + `. Est cost: SGD` + **estCost** + `.`
521. Under the approval, add a **nested Condition**: — Left value: **Dynamic content** → **Outcome** — Operator: `is equal to` — Right value: type `Approve` — **Nested `True`:** add **Send an email (V2)** → **To: requesterEmail**, **Subject:** `Purchase APPROVED:` + **item**, **Body:** confirm the request is approved. — **Nested `False`:** add **Send an email (V2)** → **To: requesterEmail**, **Subject:** `Purchase REJECTED:` + **item**, **Body:** explain the request was rejected.

In the `False` branch (auto-approved, ≤ 500):

522. Select **+** → **Add an action** → **Send an email (V2)** → **To: requesterEmail**, **Subject:** `Purchase AUTO-APPROVED:` + **item**, **Body:** confirm the request is approved automatically.

Note: ⚠ **Warning:** The **Assigned to** approver MUST be a real user in **this** tenant — pick from the people-picker dropdown so it becomes a **person chip**. A typed external email makes the run fail with *"valid users in the organization."*

Note: ⚠ **Warning:** Insert **requesterEmail**, **item**, **Outcome** etc. as **single-value** dynamic content. If the designer wraps a Send an email in an **"Apply to each"** loop, you picked a list/array output — delete the For each and re-add the plain action using the single-value tokens (Outcome and the trigger inputs are all single-value).

Note: **Tip on status updates:** to flip the row to Approved/Rejected later, use Excel **Update a row** with a **key column** (e.g. add an `ID` column set to a unique value when the row is created, then update by that key). Status updates are optional for this lab — focus on the approve + notify flow.

Step 5: Return a result and wire inputs (~10 minutes)

523. Under the branches, select **+** → **Add an action** → **Respond to the agent**. Add a **Text** output named `result` with value `Your request has been submitted for processing.`
524. Select **Publish** — publishing saves and publishes the agent flow in one step — then **Go back to agent**.
525. In the topic's tool node, map every flow input to the matching agent variable: `requester`, `requesterEmail`, `item`, `quantity`, `reason`, `estCost`.
526. After the tool node, add a **Send a message** node showing `{result}` so the user sees the confirmation.
527. Select **Save** on the topic, then **Publish** the agent.

Note: ⚠ **Warning:** If **Send an email** returns **"Unauthorized"**, reconnect the **Office 365 Outlook** connection with a **mailbox-enabled** account. All connections must show a green ✓ before the flow can run.

Step 6: Test all paths (~8 minutes)

528. Open the **Test** pane in Copilot Studio. Run a request **under** the threshold: — item `Notebook`, quantity `5`, reason `Stationery`, estCost `120` — Expect: a `Pending` row logged, an **AUTO-APPROVED** email, and the confirmation message in chat.
529. Run a request **over** the threshold and **approve** it: — item `Laptop`, quantity `2`, reason `New hires`, estCost `3000` — Respond in the **Approvals hub** (left menu of make.powerautomate.com — look under **More** if it is not pinned) → **Approve**. Expect an **APPROVED** email.
530. Run another over-threshold request and **Reject** it → expect a **REJECTED** email.
531. Open **My flows** → **run history** and the **Procurement Log** to confirm each path behaved correctly.

Note: **Tip:** Approval emails can be slow or land in **Junk**. The **Approvals hub** in the left menu is the reliable place to approve or reject pending requests.

Checkpoint

- ✅ Agent captures the request and calls the flow with all inputs mapped
- ✅ Request logged to Excel as **Pending** with a clean timestamp
- ✅ Threshold Condition routes **auto-approve** vs **manager approval**
- ✅ Approvals assigned to a **real tenant user**; requester notified of every outcome

Troubleshooting

Problem	Solution
Run fails: "valid users in the organization"	**Assigned to** must be a real tenant user picked from the people-picker dropdown (a person chip) — not a typed external email.
Condition errors on estCost	Ensure `estCost` (and `quantity`) are **Number** variables/inputs and you enter digits only.
An "Apply to each" wrapped my email action	You picked a list/array output. Delete the For each and re-add the plain action using single-value tokens (**Outcome**, **requesterEmail**, etc.).
"Unauthorized" on Send an email	Reconnect the **Office 365 Outlook** connection with a mailbox-enabled account; connections must be green ✓.
Date shows raw text / ########	Re-enter the date via the **fx** editor so it is a token; widen the Excel column to fix ########.
Approval never resumes	Respond in the **Approvals hub**; check **Junk**; emails can be slow.
Inputs empty in the flow	Re-map each agent variable to the matching flow input in the tool node.

Key Takeaways

- This is the full loop: **capture (agent)** → **log** → **approve** → **notify**.
- Approvals only run when **Assigned to** is a real tenant user picked from the dropdown.
- **Conditions** implement real business rules such as value thresholds.
- Keep single-value tokens out of accidental For each loops, and use the **fx** editor for dates.

Duration

~50 minutes

Next Steps

Proceed to Lab 15: Order Processing Workflow.

Lab 15: Order Processing Workflow (Agent + Flow)

Lab Title

End-to-End Order Processing — Agent Captures the Order, Flow Logs It, Confirms It, and Alerts the Warehouse

Lab Objectives

By the end of this lab, you will be able to:

532. Build a Copilot Studio **Order Assistant** agent that captures a customer order as structured data (name, email, product, quantity, address)
533. Connect that agent to a Power Automate **agent flow** using **When an agent calls the flow** and **Respond to the agent**
534. **Log** each order into an Excel table (**Order Log** workbook → **OrderTable**) with an automatic timestamp and a `Received` status
535. Use an **AI prompt** to draft a friendly order-confirmation email, then **Send an email** to the customer
536. Use a **Condition** so that large orders (quantity over 100) trigger a **restock alert** email to the warehouse — and small orders do not

Prerequisites

- Completed Day 2 Labs 6–11 (create an agent, capture variables, connect an agent to a flow, AI prompts)
- Completed Day 1 Labs 1–2 (email + Excel actions)
- Environment **Course Sandbox** with a working Outlook connection

Note: **Tip:** This lab reuses the exact same skills as the Procurement workflow in **Lab 14** — you are simply applying the "capture → log → generate → notify → branch" pattern to a brand-new business domain: **Order Processing**.

Scenario

You work for **ACME Pte Ltd**. A customer places an order by chatting with an agent. The moment the order is captured, the workflow should do four things automatically:

537. **Record** the order in a shared Excel order register.
538. **Confirm** the order to the customer by email.
539. **Check** the quantity — if it is a large order, **alert the warehouse** to restock.
540. **Tell the agent** it is done, so the agent can thank the customer.

You will build this as a Copilot Studio **agent** (the part the customer talks to) plus a Power Automate **agent flow** (the part that does the back-office work).

Step-by-Step Guide

Step 1: Prepare the Order Log Excel table (~5 min)

The flow needs somewhere to write each order. We use an Excel **Table** (not just a sheet) because Power Automate can only add rows to a named table.

541. Go to **OneDrive** (office.com → OneDrive) and create a new Excel workbook named exactly **Order Log**.
542. In **row 1**, type these seven column headers, one per cell, starting in cell **A1**:

`Date` | `Customer` | `Email` | `Product` | `Quantity` | `Address` | `Status`

543. Select the header cells **A1:G1**, then go to the **Insert** ribbon → click **Table**. In the dialog, make sure **My table has headers** is ticked, then click **OK**.
544. With the table selected, open the **Table Design** ribbon (top of the screen). In the **Table Name** box on the left, type **OrderTable** and press **Enter**.
545. Close Excel. Your changes save automatically to OneDrive.

Note: **Tip:** Later, if a date cell shows `########`, the column is just too narrow — widen it by double-clicking the border between the column letters. The data is fine.

Step 2: Create the Order Assistant agent (~10 min)

546. Open **Copilot Studio** (copilotstudio.microsoft.com) and confirm the environment picker (top-right) shows **Course Sandbox**. The agent and the flow must live in the **same environment**.
547. Click **Create** (left menu) → **New agent** → at the top of the setup screen click **Skip to configure**.
548. Fill in: — **Name:** `Order Assistant` — **Instructions:**

```
    You are an Order Assistant for ACME Pte Ltd.
    Collect a customer's order: customer name, email, product, quantity, and
delivery address.
    Collect one item at a time, be friendly and concise, then confirm the
order has been placed.
```

549. Click **Create**.
550. In the left menu open **Topics** → **+ Add a topic** → **From blank**. On the toolbar select **Details** and set the **Name** to `New Order`.
551. Set the topic **trigger** (latest Copilot Studio): — **Generative orchestration (default):** the **Trigger** node reads **The agent chooses**. In **Details**, set the **Description** to `Use this topic when a customer wants to place an order or buy a product. It collects the customer's name, email, product, quantity, and delivery address.` No phrases needed. — **(Optional) exact phrases:** hover the **Trigger** node → **Change trigger** → **User says a phrase**, then add `I want to place an order`, `new order`, `I'd like to buy`, `order products` (one per line).
552. Below the trigger, select the **Add node** icon (**+**) → **Ask a question** five times to add five questions. For each one, type the message, set what to **Identify**, and save the answer to a **variable**:

Question message	Identify	Save answer as variable
`What is your name?`	User's entire response	`customerName`
`What is your email?`	User's entire response	`email`
`Which product would you like to order?`	User's entire response	`product`
`How many units?`	**Number**	`quantity`
`What is the delivery address?`	User's entire response	`address`

Note: ⚠ **Warning:** For the quantity question you **must** change **Identify** to **Number**. If it stays as text, the "greater than 100" comparison in Step 6 will not work correctly.

553. Click **Save** (top-right).

Step 3: Start the agent flow (~10 min)

554. Still inside the **New Order** topic, click the **+** below your last question → **Add a tool** → **New agent flow**. This opens the Power Automate flow designer in a new tab.
555. The first (trigger) node is already **When an agent calls the flow**. Click it to open it.
556. Click **+ Add an input** once for each field below. Choose the type, then type the exact name: — **Text** → `customerName` — **Text** → `email` — **Text** → `product` — **Number** → `quantity` — **Text** → `address`

Note: **Tip:** These input names are how the agent will hand its variables to the flow. Spell them exactly as shown — you will match them up again in Step 7.

Step 4: Log the order to Excel (~5 min)

557. Click **+** → **Add an action** → search **Excel Online (Business)** → choose the action **Add a row into a table**.
558. Fill in the action's first three boxes from the dropdowns: — **Location:** OneDrive for Business — **Document Library:** OneDrive — **File:** browse to **Order Log.xlsx** — **Table: OrderTable**
559. The seven table columns now appear as fields. Map them: — **Date:** click inside the box → click the **fx** (function) icon to open the formula editor → type exactly:

```
formatDateTime(utcNow(),'yyyy-MM-dd HH:mm')
```

then click **Add** / **OK**. The formula collapses into a single token. **Never type this expression directly into the field** — always enter it through the **fx** editor, or it will be saved as plain text.

- **Customer:** dynamic content `customerName`
- **Email:** dynamic content `email`
- **Product:** dynamic content `product`
- **Quantity:** dynamic content `quantity`
- **Address:** dynamic content `address`
- **Status:** type the literal word `Received`

Step 5: Draft and send the order confirmation (~10 min)

560. *(Recommended)* Add an AI step: **+** → **Add an action** → search **Run a prompt** → choose **Run a prompt** (AI Builder — this action was formerly named *Create text with GPT using a prompt*). In the prompt box, type:

```
   Write a short, friendly order confirmation email (max 100 words).
   Customer: {customerName}; Product: {product}; Quantity: {quantity}; Delivery
to: {address}.
   Confirm the order is received and will be processed. Sign off as "The ACME
Order Team".
   Output only the email body.
```

(Insert each `{...}` value by picking it from the **dynamic content** list rather than typing it.)

561. **+** → **Add an action** → search **Office 365 Outlook** → choose **Send an email (V2)**. Fill in: — **To:** dynamic content `email` — **Subject:** type `Order confirmation:` then add dynamic content `product` — **Body:** add the AI prompt's **Text** output (the generated text from step 1). If you skipped the AI step, type your own short message using the dynamic content tokens.

Note: ⚠ **Warning:** If the Send an email action shows **Unauthorized** (a red error), the Outlook connection is signed in with the wrong account. Select the action and, at the bottom of its configuration pane (next to **Connected to …**), select **Change connection** → **+ Add new**, then sign in with a real **mailbox-enabled** tenant account. Every connection on the action should show a green check mark ✓ before you continue. (Same fix as Lab 1.)

Step 6: Add the restock-alert Condition (~10 min)

Large orders should warn the warehouse; small orders should not.

562. **+** → **Add an action** → search **Condition** → choose **Condition** (under Control).
563. Build the test: — Left box: dynamic content `quantity` — Operator: **is greater than** — Right box: type `100`
564. In the **If yes** branch (this runs only for large orders): — Click **Add an action** → **Send an email (V2)**. — **To:** the warehouse address — for testing, use your own email so you can see it arrive. — **Subject:** type `RESTOCK ALERT:` then add dynamic content `product`. — **Body:** type a message and drop in the tokens, e.g. — `Large order received: quantity units of product for customerName . Please check stock and restock if needed.`
565. Leave the **If no** branch **empty** — small orders need no alert.

Note: **Tip:** If Power Automate ever wraps an action in an automatic **For each** loop, it means you selected a field that can return multiple values. Here every field (`quantity`, `product`, etc.) is a single value, so no loop should appear. If one does, delete the action and re-add the dynamic content carefully.

Step 7: Return a result and wire the inputs (~10 min)

566. **+** → **Add an action** → search **Respond to the agent** → add it. Add a **Text** output named `result` with the value:

```
Your order has been placed and confirmed by email.
```

567. Click **Publish** — publishing saves and publishes the agent flow in one step — then close the flow tab and return to the agent (**Go back to agent** / the Copilot Studio tab).
568. In the **New Order** topic, click your flow's tool node. You will see the five flow inputs. Map each one to the matching agent **variable** from the dropdown: — `customerName` → `customerName` — `email` → `email` — `product` → `product` — `quantity` → `quantity` — `address` → `address`
569. Below the tool node, click **+** → **Send a message** and type:

```
{result} Thank you for your order!
```

(Insert `result` from the flow's outputs.)

570. Click **Save**.

Step 8: Test end-to-end (~10 min)

571. Open the **Test** pane (top-right). Click the **refresh** icon so it picks up your latest changes.
572. Type `new order` and answer the questions for a **large** order: — Name: `Mei Ling` — Email: **your own email address** — Product: `Air Fryer Pro` — Quantity: `150` — Address: `123 Orchard Rd`
573. Confirm all four results: — The agent shows your "Thank you for your order!" message. — A **new row** appears in **Order Log.xlsx** with **Status** = `Received` and a timestamp in the **Date** column. — The **order confirmation** email arrives in your inbox. — Because `150` is greater than `100`, a **RESTOCK ALERT** email **also** arrives.
574. Run it once more with a **small** quantity (e.g. `5`) and confirm: — The order is still logged and confirmed. — **No** restock alert email is sent.

Checkpoint

You are done when:

- ✅ The **Order Assistant** captures all five order fields (and quantity is a **Number**)
- ✅ The agent flow **logs** the order, **confirms** by email, and **conditionally** alerts the warehouse
- ✅ A **large** order (150) gets a restock alert; a **small** order (5) does not
- ✅ A new row with **Status = Received** and a real timestamp appears in **OrderTable**

Troubleshooting

Problem	Solution
Flow does not appear in the topic	Refresh the page; make sure the flow is **Published** and in the **same** environment (**Course Sandbox**).
Flow inputs arrive empty	In the tool node, re-map each agent variable to the matching flow input (Step 7).
Restock alert never fires (or always fires)	The Condition must compare `quantity` **is greater than** `100`, and `quantity` must be identified as a **Number** in Step 2.
Send an email shows **Unauthorized**	Reconnect Outlook with a mailbox-enabled account; confirm every connection shows a green ✓.
Date column shows the literal text `formatDateTime(...)`	You typed the expression instead of entering it through the **fx** editor. Re-enter it via **fx**.
Date cell shows `########`	The column is too narrow. Widen it — the data is correct.
An action got wrapped in **For each**	You selected a multi-value field. Remove the action and re-add single-value dynamic content.

Key Takeaways

- Order Processing is a complete **capture (agent)** → **log** → **confirm** → **alert** loop, end to end.
- A **Condition on quantity** turns a business rule (the restock threshold) into automatic behaviour.
- The timestamp comes from an **fx expression** (`formatDateTime(utcNow(),'yyyy-MM-dd HH:mm')`) entered through the formula editor, never typed as text.
- This is the **same pattern as Procurement (Lab 14)** applied to a new domain — proof that one set of skills covers many business workflows.

Duration

~55 minutes

Next Steps

Proceed to the Lab 16: Capstone Workshop to design and build your own end-to-end workflow.

Lab 16: Capstone Workshop — Build Your Own End-to-End Workflow

Lab Title

Business Workflow Workshop: Design and Build a Complete Automation

Lab Objectives

By the end of this workshop, you will be able to:

575. **Design** an end-to-end workflow for a real business scenario on paper before you build
576. **Build** it by combining a Copilot Studio agent with one or more Power Automate flows
577. **Orchestrate** triggers, actions, conditions, approvals, and notifications into one working process
578. **Test** the happy path and every branch using a structured test log
579. **Present** your workflow and explain its business value

Prerequisites

- Completed all Day 1–3 labs (you will reuse skills from across the whole course)
- Your environment **Course Sandbox** set up, with working Outlook and Excel connections (all green ✓)

Note: **Tip:** This is the last-afternoon capstone. You are not learning a new technique here — you are combining the pieces you already practised into one workflow that is **yours**. Pick a scenario you genuinely care about and the rest gets easier.

Scenario

You work for **ACME Pte Ltd**. Choose **one** real business process from your day-to-day work (or one of the four tracks below), then design and build a complete automation for it: something **triggers** it, data is **captured** and **logged**, a **business rule** decides what happens, the right people are **notified**, and the process ends in a clear **final state**.

You will work **individually or in a small group** and finish with a **working, tested workflow** plus a short **demo**.

Step-by-Step Guide

Step 1: Choose a track (~10 min)

Pick **one** scenario below (or adapt one to your own job). Each maps directly to skills you have already practised.

Track A — Sales

Lead-to-CRM workflow. A customer submits an enquiry (via the **Sales Enquiry Assistant** or an email). The workflow logs the lead, generates a personalised acknowledgement with an **AI prompt**, and notifies the assigned salesperson. High-value leads (quantity or deal size over a threshold) also raise a manager alert.

Track B — Finance

Expense / invoice approval workflow. An expense or invoice is submitted (agent form or file upload). It is logged, routed for **approval** based on amount, and the requester is

notified. Approved items move to a "to-pay" status; rejected items are returned with a reason.

Track C — Procurement

Purchase-request workflow. Staff raise a purchase request via an agent. It is logged, routed for **manager approval** when it exceeds a cost threshold, and the requester is notified. The procurement team gets a consolidated notification for approved items.

Track D — Order Processing

Order intake workflow. A customer places an order (an agent captures product, quantity, address). The order is logged, an order-confirmation email is generated with an **AI prompt**, large orders trigger a **restock alert** to the warehouse, and the customer is notified of the order status.

Step 2: Design on paper first (~20 min)

Use the orchestration method from Module 4. Complete this **design sheet** *before* you build anything — a clear plan makes the build fast and the testing obvious.

Design item	Your answer
One-sentence summary ("When ___, do ___, then ___, finally ___")	
Trigger (email / form / file upload / agent call / schedule)	
Data to capture (the fields / variables, and their types)	
Actions in order (log, generate, approve, notify…)	
Decision point(s) (the condition and the two paths)	
End states (final status + who is notified for each outcome)	
Tools used (agent? which flows? Excel? Outlook? Approvals?)	

Note: ⚠ **Warning:** Get this design sheet checked by a facilitator or peer **before** building. Fixing a plan on paper takes a minute; fixing a half-built flow takes much longer.

Step 3: Build it incrementally (~60–90 min)

Build **one piece at a time and test after each piece** — never build everything and test once at the end.

Suggested build order:

580. **Data store** — create the Excel log as a named **Table** (include any status / key columns your end states need).
581. **Capture** — build the agent topic (**Ask a question** nodes → variables) *or* configure the automated trigger (email / file). Remember to set numeric fields to **Number**.

582. **Log** — add **Add a row into a table**; test that one record is written correctly. Use the **fx** editor for any timestamp: `formatDateTime(utcNow(),'yyyy-MM-dd HH:mm')` (enter it via **fx**, never type the expression into the field).
583. **Generate** (if used) — add an **AI prompt** action; test that the generated text reads well.
584. **Decide** — add **Condition(s)** for your business rule; test **both** branches (just above and just below the threshold).
585. **Approve** (if used) — add **Start and wait for an approval**; test **approve** and **reject**.
586. **Notify** — add **Send an email** actions for each outcome; test that each one is delivered.
587. **Connect** — if using an agent, add **When an agent calls the flow** as the trigger and **Respond to the agent** at the end, then wire the agent variables → flow inputs and show the returned confirmation message.

Note: ⚠ **Warning — green connections:** Before testing any **Send an email** or Excel action, confirm every connection shows a green check mark ✓. An **Unauthorized** error on Send an email means the Outlook connection is the wrong account — reconnect with a real **mailbox-enabled** tenant account.

Note: ⚠ **Warning — approvals:** In **Start and wait for an approval**, set **Assigned to** by **picking a real tenant user from the dropdown**. Do not type an external email address — external approvers will never receive the request and the flow will appear to hang.

Note: **Tip:** If an action gets wrapped in an automatic **For each** loop, you picked a multi-value field. Most of your fields are single values — remove the action and re-add the dynamic content carefully.

Reuse your earlier labs as templates:

- Email sending → Lab 1
- Excel logging → Lab 2
- Approval + condition → Lab 3
- Scheduled / recurring trigger → Lab 4
- Form submission trigger → Lab 5
- Agent fundamentals & instructions → Lab 6
- Knowledge / RAG grounding → Lab 7
- Tools & actions → Lab 8
- Agent capture (structured data) → Lab 9
- Agent → flow integration → Lab 10
- AI-generated response → Lab 11
- Email/file trigger end-to-end → Labs 12–13
- Full approval loop → Lab 14
- Agent-driven order processing → Lab 15

Step 4: Test thoroughly (~20 min)

Complete this **test log**. Your workflow is not done until every relevant row passes. (Skip the approval rows only if your track has no approval step.)

Test case	Expected result	Pass?
Happy path (typical input)	Logged + notified + correct end state	
Approval → Approved	Approved branch runs + correct email sent	
Approval → Rejected	Rejected branch runs + correct email sent	
Threshold / edge case (just above & just below)	Correct branch chosen each time	
Bad / missing input	Agent re-asks; flow does not crash	
Run history	Every step green; logged data correct	

Note: **Tip:** Open the flow's **Run history** (Power Automate → your flow → 28-day run history) to see exactly which step failed and read the error. Green = success, red = failure.

Step 5: Present your workflow (~5 min each)

Demo to the group. Cover:

588. **The business problem** and who it helps.
589. **The trigger and the steps** — walk through your one-sentence design summary.
590. **A live run** of the happy path *and* at least one branch.
591. **The business value** — time saved, errors avoided, consistency gained.
592. **What you would add next** with more time.

Assessment checklist

Your capstone is complete when:

- ✅ It has a clear **trigger** and runs end to end
- ✅ It **captures or receives** data and **logs** it to an Excel **Table**
- ✅ It includes at least one **Condition** (branching on a business rule)
- ✅ It includes an **approval** *or* an **AI-generated** response
- ✅ It **notifies** the right people for **every** outcome
- ✅ Every relevant test case in Step 4 **passes**
- ✅ All connections are green ✓ and any approver is a **real tenant user**
- ✅ You can **explain** its business value in plain language

Stretch goals (optional)

- Add a **scheduled** flow that emails a daily summary of new records.
- Use **Update a row** to keep the Excel **Status** column current at each stage (e.g. `Received` → `Approved` → `Completed`).
- Deploy the agent to **Microsoft Teams** so colleagues can use it where they already work.
- Add a **second approver** or an **escalation** if no response within a time limit.
- Post outcomes to a **Teams channel** instead of (or as well as) email.

Duration

~2–3 hours

Congratulations 🎉

You have completed the **Business Process Automation with Power Automate and Copilot Studio Agents** course. You can now design and build end-to-end automations that capture requests, apply business rules, route approvals, and notify stakeholders — across **Sales, Finance, Procurement, and Order Processing**.

Where to go next:

- Explore the Power Automate Templates Gallery for ready-made patterns.
- Review Copilot Studio documentation for advanced agent features.
- Identify one manual process at your workplace and automate it this week.
